ABSTRACT

The Smart AI-Based Automatic Timetable Generator is developed to automate academic timetable preparation and reduce the time and effort required for manual scheduling. Creating a timetable involves coordinating subjects, faculty members, classrooms, student batches, theory classes, and laboratory sessions while satisfying multiple scheduling constraints. Managing these requirements manually can be complex and may lead to scheduling conflicts.

The proposed system uses Python and a Genetic Algorithm-based scheduling approach to generate timetables from a manually prepared Excel dataset. The dataset contains academic information, faculty details, classroom resources, batch information, working days, and available theory and laboratory time slots. The system processes this information and schedules laboratory sessions first, followed by theory classes, based on the defined constraints and resource availability.

To improve scheduling consistency, the system validates timetable allocations by checking potential conflicts involving faculty members, classrooms, and time slots. After the scheduling process, the generated timetable is converted into HTML format and displayed through a web-based interface, allowing users to view the results in an organized manner.

The system provides a structured and efficient approach to academic timetable generation by combining data processing, algorithm-based scheduling, conflict validation, and web-based presentation. It reduces manual intervention and simplifies timetable preparation while ensuring that the generated schedule is based on the input data and specified constraints.

LIST OF FIGURES

Fig No.	Figure Name	Page No.
Figure 4.1	System Functionality and User Interaction	14
Figure 4.2	System Workflow	16
Figure 6.1	Execution of the Python Timetable Generator	103
Figure 6.2	Home Page	104
Figure 6.3	About the Project Page	104
Figure 6.4	Timetable Dataset Upload Page	105
Figure 6.5	Generated Master Timetable Display	105
Figure 6.6	Semester-Wise Timetable Display	106

LIST OF TABLES

Table No	Table Name	Page No
Table 4.1	Test Cases	101

CHAPTER 1

INTRODUCTION

INTRODUCTION

Timetable generation is an important administrative activity in educational institutions. It involves allocating subjects, faculty members, classrooms, laboratories, student batches, working days, and time slots according to academic requirements. A properly designed timetable ensures the smooth conduct of classes and effective utilization of available resources.

Preparing timetables manually is challenging because several constraints must be satisfied simultaneously. Faculty members may handle multiple subjects, classrooms have limited availability, and laboratory sessions require longer time slots. When multiple semesters share faculty members and resources, timetable preparation becomes more complex. Manual scheduling also requires repeated modifications to avoid faculty clashes, classroom conflicts, and overlapping academic activities.

Artificial Intelligence (AI) and optimization techniques provide methods to automate complex scheduling activities. The **Smart AI-Based Automatic Timetable Generator** is developed using Python to simplify academic timetable preparation through a Genetic Algorithm-based scheduling approach.

The system accepts a manually prepared Excel workbook containing subject details, faculty information, classroom resources, student batches, working days, and theory and laboratory time slots. Python processes the dataset and applies the defined scheduling constraints. Laboratory sessions are allocated first, followed by theory classes, ensuring that existing laboratory allocations are considered during theory scheduling.

The generated timetable is checked for potential faculty clashes, classroom conflicts, repeated subjects on the same day, and theory–laboratory overlaps according to the implemented validation rules. Finally, the timetable is generated in HTML format and displayed through the existing web interface. The system aims to reduce manual effort, improve scheduling consistency, and provide an organized approach to academic timetable generation.

1.1 Problem Statement

Academic timetable preparation involves coordinating subjects, faculty members, classrooms, laboratories, batches, and time slots. Managing these requirements manually becomes difficult when multiple semesters share the same resources.

Manual scheduling may result in faculty clashes, classroom conflicts, and overlapping theory and laboratory sessions. Laboratory allocation is particularly challenging because practical sessions require longer time blocks and specific faculty and batch arrangements. Any changes in academic requirements may also require repeated timetable modifications.

Therefore, an automated approach is required to simplify timetable preparation and reduce scheduling conflicts. The proposed Smart AI-Based Automatic Timetable Generator uses Python, an Excel dataset, and a Genetic Algorithm-based scheduling approach. It schedules laboratory sessions first, followed by theory classes, and validates the generated timetable according to the implemented constraints.

1.2 Objectives of the Project

The main objective is to automate academic timetable generation using Python and Genetic Algorithm-based scheduling.

The specific objectives are:

1. **Automate timetable generation:** Reduce manual effort and preparation time.
2. **Process academic data:** Read subjects, faculty, classrooms, batches, and time slots from Excel.
3. **Identify constraints:** Consider faculty, classroom, laboratory, and time-slot requirements.
4. **Apply Genetic Algorithm:** Explore suitable timetable arrangements.
5. **Schedule laboratories first:** Allocate lab sessions before theory classes.
6. **Generate theory timetables:** Assign subjects to available time slots.
7. **Minimize conflicts:** Reduce faculty clashes, room conflicts, and overlapping sessions.
8. **Validate timetables:** Check the generated schedule against implemented rules.
9. **Provide web-based output:** Display the timetable in HTML format.
10. **Improve resource utilization:** Support effective use of faculty, classrooms, and laboratories.

CHAPTER 2

LITERATURE SURVEY

LITERATURE SURVEY

Chen et al. [1] examined university course timetabling problems and the different optimization techniques used to solve them. Their study discussed scheduling constraints, metaheuristic algorithms, and hybrid optimization approaches for generating feasible academic timetables. The research highlighted the complexity of coordinating courses, faculty members, and available resources. This study provides a foundation for selecting suitable optimization techniques for the proposed Smart AI-Based Automatic Timetable Generator.

Algethami and Laesanklang et al.[2] studied mathematical modelling for course timetabling with a focus on faculty-course assignment constraints. Their approach considered the allocation of courses to appropriate faculty members while satisfying academic scheduling requirements. The study highlighted the importance of preventing faculty clashes and maintaining consistent course assignments. This research is relevant to the proposed system, which considers faculty assignments during timetable generation.

Davison et al. [3] investigated university course timetabling under hybrid teaching conditions. Their study considered different teaching arrangements and the constraints involved in organizing academic activities. The research demonstrated the importance of incorporating institutional requirements into scheduling models. This study provides useful insights into adapting automatic timetable generation systems to different teaching formats and scheduling conditions.

Steiner et al. [4] studied curriculum-based university course timetabling while considering individual student study programmes. Students may follow different combinations of courses, making it challenging to schedule classes without conflicts. The research addressed the need to coordinate course schedules with curriculum requirements. This approach is relevant to the proposed system, which handles multiple semesters, subjects, and student batches.

Müller et al. [5] examined real-world university course timetabling through the International Timetabling Competition 2019. Their study highlighted the practical difficulties of generating timetables that satisfy multiple institutional constraints. The research emphasized the importance of considering available resources, course requirements, and timetable feasibility. These findings provide useful background for developing an automated system that addresses practical academic scheduling requirements.

Mokhtari et al. [6] investigated university course timetabling using mathematical modelling and a case study. Their research considered multiple academic requirements and scheduling objectives while formulating the timetable generation problem. Mathematical modelling helps represent scheduling constraints systematically and evaluate possible arrangements. This study is relevant to the proposed system because academic requirements must be translated into rules that can be processed during automated timetable generation.

González López et al. [7] explored the use of Genetic Algorithms for generating personalized university timetables. Their approach considered course requirements and student preferences while searching for suitable timetable arrangements. Genetic Algorithms use evolutionary operations such as selection, crossover, and mutation to explore candidate solutions. This research is relevant to the proposed project, which uses a Genetic Algorithm-based approach to generate timetables according to predefined scheduling constraints.

Bashab et al. [8] reviewed optimization techniques used in university timetabling problems. Their study discussed scheduling constraints, solution methodologies, benchmark problems, and challenges associated with existing approaches. The research provided an overview of how different optimization techniques can be applied to complex scheduling problems. This study supports the selection of appropriate optimization methods for timetable generation and the validation of scheduling constraints.

Abdipoor et al. [9] examined metaheuristic approaches for solving university course timetabling problems. These methods explore different candidate solutions to identify suitable schedules for problems involving multiple constraints. Their research highlighted the role of optimization techniques in addressing the complexity of academic scheduling. Since Genetic Algorithms belong to the broader category of metaheuristic optimization methods, this study provides relevant background for the scheduling approach adopted in the proposed system.

Mallari et al. [10] studied the synchronization of university coursework schedules using an optimization-based approach. Their research considered academic calendar requirements and restrictions involving students, instructors, workloads, and institutional policies. Coordinating these factors is important when multiple courses must be scheduled within the same academic period. This study is relevant to the proposed Smart AI-Based Automatic Timetable Generator, which coordinates academic activities while considering faculty assignments and available time slots.

CHAPTER 3

SYSTEM ANALYSIS

SYSTEM ANALYSIS

System analysis examines the existing timetable preparation process, identifies its challenges, and determines the requirements of the proposed system. It focuses on improving academic scheduling through automation.

3.1 Existing System

In many educational institutions, academic timetables are prepared manually using spreadsheets or basic scheduling tools. Coordinators collect information about subjects, faculty members, classrooms, laboratories, batches, and available time slots before assigning classes.

Manual preparation requires careful coordination because faculty availability, classroom allocation, laboratory duration, and subject distribution must be considered. When multiple semesters share the same resources, the process becomes more complicated and time-consuming.

The existing system may result in faculty clashes, classroom conflicts, and overlapping theory and laboratory sessions. Changes in faculty assignments or academic requirements also require repeated timetable modifications and manual checking.

3.2 Proposed System

The proposed Smart AI-Based Automatic Timetable Generator automates timetable preparation using Python and a Genetic Algorithm-based scheduling approach. It accepts a manually prepared Excel dataset containing subject details, faculty information, classrooms, batches, working days, and theory and laboratory time slots.

The Python program processes the dataset and applies the defined scheduling constraints. Laboratory sessions are allocated first, followed by theory classes using the Genetic Algorithm-based approach. The generated timetable is then validated for faculty clashes, classroom conflicts, repeated subjects on the same day, and theory–laboratory overlaps according to the implemented rules.

Finally, the timetable is generated in HTML format and displayed through the existing web interface. Users can view the combined Master Timetable and individual semester-wise schedules.

The major features of the proposed system are:

1. **Excel Dataset Input:** Accepts academic and resource information through an Excel workbook.
2. **Automated Scheduling:** Generates timetables using Python and a Genetic Algorithm-based approach.
3. **Constraint-Based Allocation:** Considers faculty, classrooms, batches, laboratories, and time slots.
4. **Laboratory-First Scheduling:** Allocates laboratory sessions before theory classes.
5. **Conflict Validation:** Checks the timetable against the implemented scheduling rules.
6. **Web-Based Output:** Displays the generated timetable in HTML format.
7. **Semester-Wise Display:** Provides combined and individual semester timetable views.
8. **Reduced Manual Effort:** Reduces repetitive scheduling and correction work.

3.3 Hardware and Software Requirements

The system requires suitable hardware and software to process the Excel dataset, execute the scheduling program, and display the generated timetable.

3.3.1 Hardware Requirements

- **Processor:** Intel Core i3 or equivalent processor.
- **RAM:** Minimum 4 GB for running Python and processing the dataset.
- **Storage:** At least 500 MB of free space for project files, libraries, and generated output.
- **Input Devices:** Keyboard and mouse for data entry and system operation.
- **Display:** Laptop or desktop display for viewing the timetable and web interface.

3.3.2 Software Requirements

- **Operating System:** Windows 10 or later, or another compatible operating system.
- **Python:** Used for dataset processing, scheduling, validation, and HTML generation.
- **Pandas:** Used to read and process Excel dataset information.
- **OpenPyXL:** Supports reading Excel (.xlsx) files when required.
- **Microsoft Excel:** Used to prepare and maintain the input dataset.
- **HTML and CSS:** Used for displaying the generated timetable through the existing webpage.
- **Web Browser:** Google Chrome or Microsoft Edge for viewing the timetable.
- **Code Editor:** Visual Studio Code or Python IDLE for writing and executing the Python program.

CHAPTER 4

METHODOLOGY AND SYSTEM DESIGN

METHODOLOGY AND SYSTEM DESIGN

The methodology explains the steps followed to develop the Smart AI-Based Automatic Timetable Generator. It covers data collection, preprocessing, algorithm development, evaluation, and deployment. The system uses Python to process a manually prepared Excel dataset and a Genetic Algorithm-based optimization approach to generate timetables. Laboratory sessions are scheduled first, followed by theory classes. The final timetable is validated and displayed in HTML format through the web interface.

4.1 Methodology

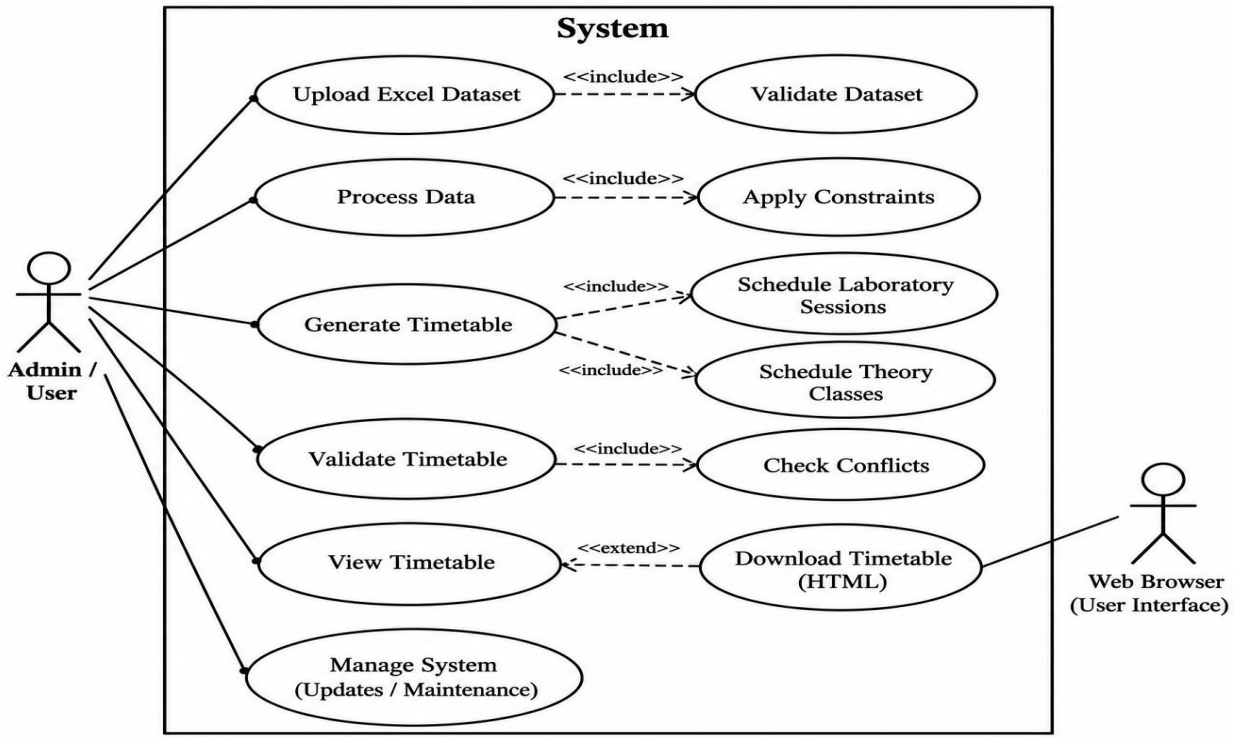


Figure 4.1: System Functionality and User Interaction

1. Data Collection: Academic information is collected through an Excel dataset containing subjects, faculty details, classrooms, batches, working days, and theory and laboratory time slots.

2. Data Preprocessing: The Python program reads and organizes the Excel data, checking the required information and preparing it for timetable generation.

3. Algorithm Selection and Development: A Genetic Algorithm-based optimization approach is implemented to generate timetables based on the defined scheduling constraints. Laboratory sessions are allocated first, followed by theory classes.

4. Model or Algorithm Evaluation: The generated timetable is validated for faculty clashes, classroom conflicts, repeated subjects in a day, and theory–laboratory overlaps according to the implemented rules.

5. Deployment: Python processes the Excel dataset and generates the timetable in HTML format, which users can view through a web browser. The current implementation runs locally and is not publicly hosted online.

4.2 System Design

System design explains the main components of the Smart AI-Based Automatic Timetable Generator and how they interact. The system consists of Excel dataset input, data processing, constraint handling, laboratory and theory scheduling, conflict validation, and HTML output components.

The input component provides academic and resource information. The Python program processes the data, applies scheduling constraints, and allocates laboratory sessions followed by theory classes. The generated timetable is validated and presented in HTML format through the web interface.

4.3 System Architecture

The system architecture shows how the components interact, from processing the Excel dataset using Python to scheduling and validating the timetable before generating the final HTML output.

Major Components:

1. **Dataset Input:** Accepts the manually prepared Excel workbook containing subject details, faculty information, classroom details, student batches, working days, and available theory and laboratory time slots.
2. **Data Processing:** Reads the Excel dataset using Python and organizes the academic and resource information into a suitable format for timetable generation.

3. **Constraint Handling:** Identifies and applies scheduling rules related to faculty availability, classroom allocation, batch requirements, time slots, and subject distribution.
4. **Laboratory Scheduling:** Allocates laboratory sessions based on the required duration, available time slots, batch arrangements, faculty assignments, and classroom availability.
5. **Theory Scheduling:** Assigns theory classes using the Genetic Algorithm-based optimization approach while considering the defined scheduling constraints and available resources.
6. **Conflict Validation:** Checks the generated timetable for faculty clashes, classroom conflicts, repeated subjects on the same day, and theory–laboratory overlaps according to the implemented rules.
7. **Output Generation:** Combines the scheduled theory and laboratory sessions and generates the final timetable in HTML format for easy viewing.
8. **Web Interface:** Provides an interface for uploading the Excel dataset and viewing the generated master timetable and semester-wise timetables through a web browser.

4.4 System Workflow

The system workflow describes the sequence of operations followed to generate the final academic timetable. It begins with preparing the input dataset and ends with displaying the generated timetable through the web interface.

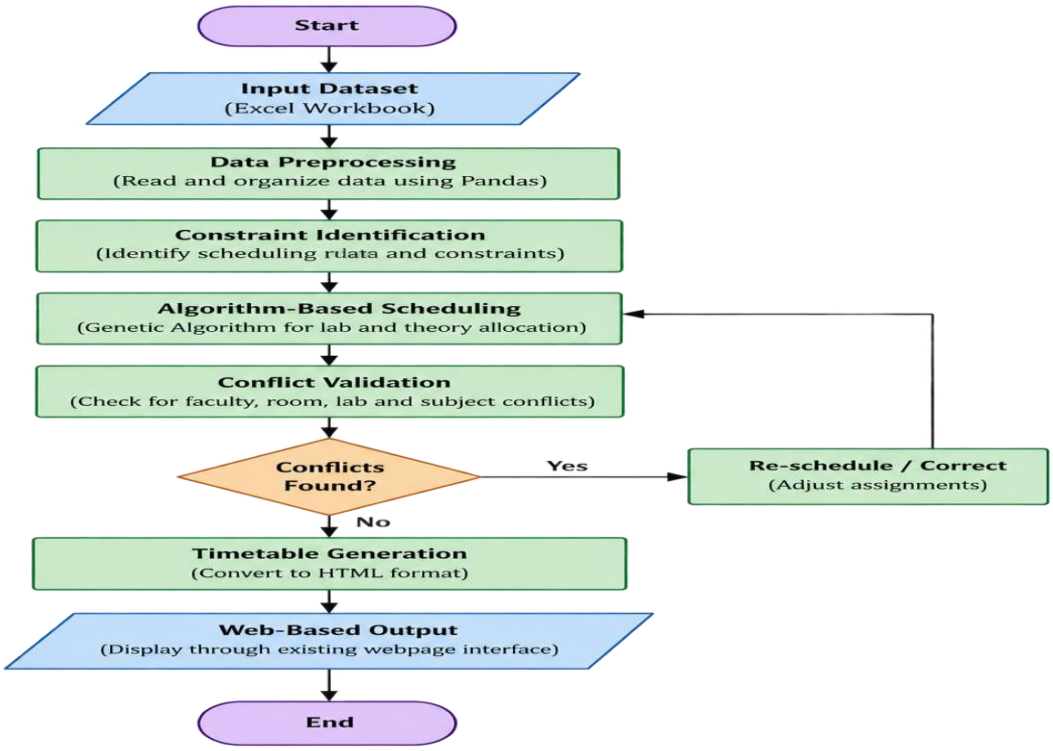


Figure 4.2: System Workflow of the Smart AI-Based Automatic Timetable Generator

The system workflow of the Smart AI-Based Automatic Timetable Generator begins with the preparation of an Excel workbook containing essential academic information, such as subject details, faculty information, classrooms, laboratory details, student batches, and available time slots. The system reads and processes this dataset using Python and the Pandas library. During data preprocessing, the information is organized into a structured format for further processing. Next, the system identifies the scheduling rules and constraints, including faculty availability, classroom availability, laboratory requirements, subject allocation, and time-slot restrictions. After identifying these constraints, the algorithm-based scheduling process begins. A Genetic Algorithm (GA) is used to allocate laboratory sessions first, followed by theory classes, while attempting to minimize scheduling conflicts and ensure proper resource utilization.

CHAPTER 5

IMPLEMENTATION

SYSTEM IMPLEMENTATION

5.1 Programming Environment

The Smart AI-Based Automatic Timetable Generator is developed using Python to process academic data, perform timetable scheduling, validate scheduling constraints, and generate the final timetable. An Excel workbook is used as the input dataset, which is manually prepared with the required academic and resource information. The system uses a Genetic Algorithm-based approach to find suitable timetable arrangements. HTML and CSS support the presentation of the generated timetable through the existing webpage interface. The generated timetable can be viewed using a web browser.

- **Programming Language:** Python
- **Algorithm Used:** Genetic Algorithm-based scheduling
- **Data Processing:** Pandas and compatible Excel-reading libraries
- **Input Dataset:** Excel workbook (.xlsx)
- **Front-End:** HTML and CSS
- **Editor/IDE:** Visual Studio Code / Python IDLE / Any compatible code editor
- **Platform:** Windows or another compatible operating system
- **Output Format:** HTML
- **Browser:** Google Chrome / Microsoft Edge / Any compatible web browser

.

5.2 Source Code

1.Smart_tt_generator.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
import argparse
import html
import io
import itertools
import json
import math
import random
import re
import threading
import time
import webbrowser
from collections import Counter, defaultdict
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path
from urllib.parse import unquote, urlparse, parse_qs

import pandas as pd

BASE_DIR = Path(__file__).resolve().parent
HTML_NAMES = ["index.html", "mLAC Smart Timetable.html"]
CSS_NAMES = ["styles.css"]
SAMPLE_NAMES = ["tt_dataset.xlsx"]
OUTPUT_FILE = BASE_DIR / "Generated_Timetable.html"
PORT = 8765
# ---------------------------------------------------------------------------
# Rules that are policy rather than data
# ---------------------------------------------------------------------------
```

```
# Combined dataset rows that the timetable shows as two separate theory subjects.
PLACEHOLDER_NAMES = {"", "x", "tba", "tbd", "na", "n/a", "none", "-", "nan"}
# Semesters that also run a Project Lab (semester -> sessions per week). The dataset has no
# Project row, so this rule adds it; a "Project" row in the Academic Dataset overrides it.
PROJECT_LAB_SESSIONS = {5: 2}
HARD = 1000.0                                   # hard-violation weight

DEFAULT_THEORY = ["8:30-9:20", "9:20-10:10", "10:10-11:00", "11:00-11:50",
                  "11:50-12:40", "12:40-1:30", "1:30-2:20"]
DEFAULT_LAB = ["8:00-11:00", "11:00-2:00", "2:00-5:00"]
DEFAULT_DAYS = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"]
EMPTY = frozenset()
# ============================================================================
# Small helpers
# ============================================================================
def clean(v):
    if v is None:
        return ""
    if isinstance(v, float) and math.isnan(v):
        return ""
    return re.sub(r"\s+", " ", str(v)).strip()
def fkey(name):
    return re.sub(r"[^a-z0-9]", "", clean(name).lower())


def to_int(v, default=None):
    try:
        s = clean(v)
        if s == "":
            return default
        return int(float(s))
    except (ValueError, TypeError):
        return default
```

```
def split_faculty(cell):
    cell = re.sub(r"\band\b", "&", clean(cell), flags=re.I)
    out = []
    for p in re.split(r"[&,/;+]", cell):
        p = p.strip()
        if p.lower() not in PLACEHOLDER_NAMES:
            out.append(p)
    return out

def parse_clock(t):
    h, m = t.split(":")
    h, m = int(h), int(m)
    if 1 <= h <= 6:                    # 1:30 -> 13:30
        h += 12
    return h * 60 + m

def parse_range(s):
    s = clean(s).replace("–", "-").replace("—", "-").replace(" ", "")
    a, b = s.split("-")
    return parse_clock(a), parse_clock(b)

def pretty_range(s):
    return clean(s).replace("-", "–").replace(" ", "")

def ordinal(n):
    if 10 <= n % 100 <= 20:
        suf = "th"
    else:
        suf = {1: "st", 2: "nd", 3: "rd"}.get(n % 10, "th")
    return f"{n}{suf}"

def find_sheet(sheets, *keys, exclude=()):
```

```
    for name, df in sheets.items():
        low = name.lower()
        if all(k in low for k in keys) and not any(x in low for x in exclude):
            return df
    return None

def col(df, *names):
    cols = {str(c).strip().lower(): c for c in df.columns}
    for n in names:                    # exact first
        if n in cols:
            return cols[n]
    for n in names:                    # then substring
        for low, orig in cols.items():
            if n in low:
                return orig
    return None
# ============================================================================
# 1. DATASET -> PROBLEM
# ============================================================================
class TSub:
    """A theory subject (after splitting combined rows)."""
    def __init__(self, sem, name, teachers, periods, source):
        self.sem, self.name, self.teachers = sem, name, teachers
        self.keys = frozenset(fkey(t) for t in teachers)
        self.periods, self.source = periods, source
        self.maxpd = 1

class LSub:
    """A laboratory subject."""
    def __init__(self, sem, name, teachers, sessions, joint):
        self.sem, self.name, self.teachers = sem, name, teachers
        self.keys = [fkey(t) for t in teachers]
```

```
        self.sessions, self.joint = sessions, joint

    @property
    def lab_name(self):
        return self.name if self.name.lower().endswith("lab") else f"{self.name} Lab"

class Problem:
    pass

def read_sheets(source, filename="dataset.xlsx"):
    """source: path or bytes."""
    if isinstance(source, (bytes, bytearray)):
        low = filename.lower()
        if low.endswith(".csv"):
            raise ValueError("Please drop the Excel workbook (.xlsx) that contains all the "
                             "dataset sheets - a single CSV cannot hold them.")
        buf = io.BytesIO(source)
        try:
            return pd.read_excel(buf, sheet_name=None)
        except ImportError:
            raise ValueError("This file type needs an extra package. Please save the workbook as
.xlsx.")
    return pd.read_excel(source, sheet_name=None)

def load_problem(sheets):
    P = Problem()
    P.notes = []          # data checks shown to the user: (title, text)
    P.flag_rows = 0

    acad = find_sheet(sheets, "academic")
    res = find_sheet(sheets, "resource")
    if acad is None:
```

```
    raise ValueError("The workbook has no 'Academic Dataset' sheet.")
if res is None:
    raise ValueError("The workbook has no 'Resource Dataset' sheet.")
batch_df = find_sheet(sheets, "batch")
fac_df = find_sheet(sheets, "faculty", exclude=("academic",))
th_df = find_sheet(sheets, "theory", "slot")
lb_df = find_sheet(sheets, "lab", "slot")
day_df = find_sheet(sheets, "day")

# ---- days / slots ----------------------------------------------------
P.days = DEFAULT_DAYS
if day_df is not None:
    c = col(day_df, "day")
    d = [clean(x) for x in day_df[c].tolist() if clean(x)] if c is not None else []
    if d:
        P.days = d

def slot_list(df, default):
    if df is None:
        return default
    c = col(df, "time")
    s = [clean(x) for x in df[c].tolist() if clean(x)] if c is not None else []
    return s or default

P.theory_slots = slot_list(th_df, DEFAULT_THEORY)
P.lab_slots = slot_list(lb_df, DEFAULT_LAB)
P.theory_rng = [parse_range(s) for s in P.theory_slots]
P.lab_rng = [parse_range(s) for s in P.lab_slots]
# A lab block occupies every theory period it overlaps.
P.block_slots = []
for ls, le in P.lab_rng:
    P.block_slots.append(frozenset(i for i, (ts, te) in enumerate(P.theory_rng)
```

```
                    if ts < le and ls < te))

    # ---- resources ----------------------------------------------------------
    c_type, c_name = col(res, "resource type", "type"), col(res, "resource name", "name")
    c_cap, c_use = col(res, "capacity"), col(res, "usage", "note")
    P.classrooms, P.labs, P.room_info = [], [], []
    for _, r in res.iterrows():
        nm, tp = clean(r[c_name]), clean(r[c_type]).lower()
        if not nm:
            continue
        cap = to_int(r[c_cap], 0) if c_cap is not None else 0
        use = clean(r[c_use]) if c_use is not None else ""
        if tp.startswith("lab"):
            P.labs.append(nm)
            P.room_info.append((nm, "Laboratory", cap, use))
        else:
            P.classrooms.append(nm)
            P.room_info.append((nm, "Classroom", cap, use))
    P.room_cap = {n: cap for n, _, cap, _ in P.room_info}
    if not P.classrooms:
        raise ValueError("The Resource Dataset lists no classrooms.")
    if not P.labs:
        P.notes.append(("No laboratories", "The Resource Dataset lists no laboratories, so labs cannot
be placed."))

    home, shared = {}, defaultdict(list)
    for nm, tp, cap, use in P.room_info:
        if tp != "Classroom":
            continue
        m = re.search(r"(\d)(?:st|nd|rd|th)\s*year\s*classroom", use, re.I)
        if m:
            home[int(m.group(1)) * 2 - 1] = nm
```

```
    m2 = re.search(r"available to (\d)(?:st|nd|rd|th)\s*year", use, re.I)
    if m2:
        shared[int(m2.group(1)) * 2 - 1].append(nm)
claimed = set(home.values())
P.home, P.shared = home, shared

# ---- academic dataset ------------------------------------------------
c_sem, c_sub = col(acad, "semester", "sem"), col(acad, "subject")
c_fac, c_typ = col(acad, "faculty"), col(acad, "subject type", "type")
c_mark, c_th = col(acad, "marks"), col(acad, "theory periods", "theory")
c_lreq, c_lses = col(acad, "lab required"), col(acad, "lab sessions")
c_year = col(acad, "year")
if None in (c_sem, c_sub, c_fac):
    raise ValueError("Academic Dataset needs Semester, Subject and Faculty columns.")

P.table_rows, P.theory, P.labsubs = [], defaultdict(list), defaultdict(list)
P.ignored, P.disp, P.lab_qualified = [], {}, set()
P.language_keys, P.teaches = set(), defaultdict(list)
P.n_scheduled_rows = 0

def remember(name):
    k = fkey(name)
    if k and k not in P.disp:
        P.disp[k] = name
    return k

for _, r in acad.iterrows():
    subj = clean(r[c_sub])
    sem = to_int(r[c_sem])
    if not subj or sem is None:
        continue
    teachers = split_faculty(r[c_fac])
```

```
for t in teachers:
    remember(t)
typ = clean(r[c_typ]) if c_typ is not None else ""
marks = clean(r[c_mark]) if c_mark is not None else "x"
periods = to_int(r[c_th], 0) if c_th is not None else 0
lab_req = clean(r[c_lreq]).lower() in {"yes", "y", "true", "1"} if c_lreq is not None else False
sessions = to_int(r[c_lses], 0) if c_lses is not None else 0
if lab_req and sessions <= 0:
    sessions = 1
year = clean(r[c_year]) if c_year is not None else f"{ordinal((sem + 1) // 2)} Year"

low = subj.lower()
# Language teachers are identified for LAB exclusion only.
# Their language theory subjects remain schedulable.
if any(w in low for w in LANGUAGE_WORDS):
    P.language_keys |= {fkey(t) for t in teachers}
ignored = False
flags = []
P.n_scheduled_rows += 1
if not teachers:
    flags.append(f"{subj} (Sem {sem}) has no faculty named - it will show as TBA.")
if marks == "" or marks.lower() in {"na", "n/a", "nan", "-"}:
    flags.append(f"{subj} (Sem {sem}) has no marks value.")
if flags:
    P.flag_rows += 1
    P.notes.extend((subj, f) for f in flags)

lab_txt = f"{sessions}/wk" if sessions else "—"
P.table_rows.append(dict(sem=sem, year=year, subject=subj, faculty=clean(r[c_fac]),
                         type=typ, marks=marks, theory=periods, lab=lab_txt,
                         flagged=bool(flags)))
if ignored:
```

```
            continue

        for t in teachers:
            P.teaches[fkey(t)].append((subj, sem))
        if periods > 0:
            if low in SPLIT_SUBJECTS:
                a, b = SPLIT_SUBJECTS[low]
                first = periods - periods // 2
                P.theory[sem].append(TSub(sem, a, teachers, first, subj))
                P.theory[sem].append(TSub(sem, b, teachers, periods // 2, subj))
            else:
                P.theory[sem].append(TSub(sem, subj, teachers, periods, subj))
        if lab_req and sessions > 0:
            joint = typ.strip().lower() == "lab"
            P.labsubs[sem].append(LSub(sem, subj, teachers, sessions, joint))
            for t in teachers:
                P.lab_qualified.add(fkey(t))

    # ---- project lab rule (5th semester: ML + DV + 2 Project = 4 lab blocks) ----
    for sem, n_sessions in PROJECT_LAB_SESSIONS.items():
        subs = P.labsubs.get(sem)
        if not subs or any("project" in x.name.lower() for x in subs):
            continue
        teachers, seen = [], set()
        for x in subs:
            for t in x.teachers:
                if fkey(t) not in seen:
                    seen.add(fkey(t))
                    teachers.append(t)
        P.labsubs[sem].append(LSub(sem, "Project", teachers[:2], n_sessions, True))
        P.notes.append((f"Semester {sem}: Project Lab added",
                        f"{n_sessions} Project Lab blocks per week were added by rule "
```

```
                f"(taught by {' & '.join(teachers[:2])}). Add a 'Project' row to the "
                "Academic Dataset to change this."))

# semesters, highest year = highest priority
P.sems = sorted(set(P.theory) | set(P.labsubs), reverse=True)
if not P.sems:
    raise ValueError("No schedulable subjects were found in the Academic Dataset.")
P.theory_sems = [s for s in P.sems if P.theory.get(s)]

# ---- classroom permissions per semester ------------------------------
P.rooms_for = {}
for sem in P.sems:
    lst = []
    if sem in home:
        lst.append(home[sem])
    lst += [r for r in shared.get(sem, []) if r not in lst]
    if not lst:
        lst = [r for r in P.classrooms if r not in claimed] or list(P.classrooms)
        P.notes.append((f"Semester {sem} has no assigned classroom",
                        "Using unassigned classrooms: " + ", ".join(lst)))
    P.rooms_for[sem] = lst

# ---- batches ---------------------------------------------------------
P.batches, P.strength = {}, {}
for sem in P.sems:
    P.batches[sem] = ["Batch A", "Batch B"]
if batch_df is not None:
    cs = col(batch_df, "semester", "sem")
    cn = col(batch_df, "batch names", "batch name")
    ct = col(batch_df, "total batches", "batches for lab")
    cstr = col(batch_df, "total students", "strength")
    for _, r in batch_df.iterrows():
```

```
        sem = to_int(r[cs]) if cs is not None else None
        if sem is None or sem not in P.batches:
            continue
        names = [n.strip() for n in re.split(r"[,&/;]", clean(r[cn])) if n.strip()] if cn is not None else[]
        claimed_n = to_int(r[ct]) if ct is not None else None
        if names:
            P.batches[sem] = names
        elif claimed_n:
            P.batches[sem] = [f"Batch {chr(65 + i)}" for i in range(claimed_n)]
        if names and claimed_n and claimed_n != len(names):
            P.notes.append((f"Semester {sem} batch count",
                            f"The sheet says {claimed_n} lab batches but names {len(names)}. "
                            f"Using the {len(names)} named batches."))
            P.flag_rows += 1
        if cstr is not None:
            P.strength[sem] = to_int(r[cstr])
    for sem in P.sems:
        nb = len(P.batches[sem])
        if P.labsubs.get(sem) and nb > len(P.labs):
            P.notes.append((f"Semester {sem} needs {nb} labs at once",
                            f"Only {len(P.labs)} laboratories exist; the extra batches cannot be placed."))
        st = P.strength.get(sem)
        cap = P.room_cap.get(P.rooms_for[sem][0], 0)
        if st and cap and st > cap:
            P.notes.append((f"Semester {sem} classroom capacity",
                            f"{st} students but {P.rooms_for[sem][0]} seats {cap}. "
                            "The timetable follows the room rule; confirm the room is big enough."))
            P.flag_rows += 1

    # ---- faculty sheet: availability + names -------------------------------
    P.unavail = defaultdict(set)
    P.faculty_sheet_keys = set()
```

```
    if fac_df is not None:
        cn = col(fac_df, "faculty name", "name")
        ca = col(fac_df, "availability")
        for _, r in fac_df.iterrows():
            nm = clean(r[cn]) if cn is not None else ""
            if nm.lower() in PLACEHOLDER_NAMES:
                continue
            k = remember(nm)
            P.faculty_sheet_keys.add(k)
            txt = clean(r[ca]).lower() if ca is not None else ""
            m = re.search(r"(not available|unavailable|leave|off|except)(.*)", txt)
            if m:
                for di, dn in enumerate(P.days):
                    if dn.lower() in m.group(2) or dn[:3].lower() in re.findall(r"[a-z]+", m.group(2)):
                        P.unavail[k].add(di)
        missing = sorted(P.disp[k] for k in P.teaches if k not in P.faculty_sheet_keys)
        if missing:
            P.notes.append(("Faculty missing from the Faculty sheet",
                            ", ".join(missing) + " teach(es) in the Academic Dataset but have no Faculty-sheet
entry."))
            P.flag_rows += 1

    # ---- assistant pool for labs ----------------------------------------
    lang = P.language_keys
    P.pool = sorted(k for k in P.lab_qualified if k not in lang)

    # ---- sanity checks --------------------------------------------------
    nd, ns = len(P.days), len(P.theory_slots)
    for sem in P.theory_sems:
        tot = sum(s.periods for s in P.theory[sem])
        for s in P.theory[sem]:
            s.maxpd = max(1, math.ceil(s.periods / nd))
```

```
        if s.periods > nd:
            P.notes.append((f"{s.name} (Sem {sem})",
                            f"{s.periods} periods/week but only {nd} days; the 'one per day' rule is relaxed."))
        if tot > nd * ns:
            raise ValueError(f"Semester {sem}: {tot} theory periods cannot fit in {nd * ns} weekly slots.")
    return P
# ============================================================================
# 2. LAB ROUNDS  (who does which lab, in what rotation)
# ============================================================================
class Round:
    def __init__(self, sem, entries):
        self.sem = sem
        self.entries = entries   # list of dict(batch, sub, base(list of keys), lab_room)
        self.day = self.block = None
        self.base_keys = frozenset(k for e in entries for k in e["base"])

def build_rounds(P):
    rounds = []
    for sem in P.sems:
        subs = P.labsubs.get(sem, [])
        if not subs:
            continue
        batches = P.batches[sem]
        n = len(batches)
        rot = [s for s in subs if not s.joint]
        joint = [s for s in subs if s.joint]

        def mk_joint(s):
            base = [k for k in s.keys if k not in P.language_keys]
            ents = []
```

```
    for bi, b in enumerate(batches):
        ents.append(dict(batch=b, sub=s, base=base[bi::n], bi=bi))
    return Round(sem, ents)


items = []
for k in range(max([s.sessions for s in rot] or [0])):
    items += [s for s in rot if s.sessions > k]
m = len(items)
if m and len({x.name for x in items}) < n:   # too few subjects to rotate -> run together
    for s in items:
        rounds.append(mk_joint(s))
elif m:
    best = None
    tried = 0
    orders = itertools.permutations(range(m)) if m <= 6 else iter([tuple(range(m))])
    for order in orders:
        tried += 1
        seq = [items[i] for i in order]
        rs, ok = [], True
        for r in range(m):
            ents, seen = [], set()
            for bi, b in enumerate(batches):
                s = seq[(r + bi) % m]
                base = [k for k in s.keys if k not in P.language_keys][:2]
                if seen & set(base):
                    ok = False
                seen |= set(base)
                ents.append(dict(batch=b, sub=s, base=base, bi=bi))
            rs.append(Round(sem, ents))
        if ok:
            best = rs
            break
```

```
            if tried > 800:
                break
        if best is None:                # no clash-free pairing exists -> run each together
            best = [mk_joint(x) for x in items]
        rounds += best
    for s in joint:
        for _ in range(s.sessions):
            rounds.append(mk_joint(s))
  return rounds
#=============================================================================
# 3. GA - LAB PLACEMENT
# =============================================================================
class LabPlacer:
  def __init__(self, P, rounds, rng):
    self.P, self.rounds, self.rng = P, rounds, rng
    self.nd, self.nb = len(P.days), len(P.lab_slots)
    self.cells = [(d, b) for d in range(self.nd) for b in range(self.nb)]
    self.cap = max(1, len(P.labs))
    self.ns = len(P.theory_slots)
    self.need = {}
    for sem in P.theory_sems:
      subs = P.theory[sem]
      self.need[sem] = (sum(s.periods for s in subs), sum(s.maxpd for s in subs))

  def cost(self, genes):
    P, hard, soft = self.P, 0, 0.0
    cells = defaultdict(list)
    sem_days = defaultdict(list)
    sem_blocks = defaultdict(dict)
    wednesday = next((i for i, day in enumerate(P.days) if clean(day).lower().startswith("wed")),
None)
    for r, (d, b) in zip(self.rounds, genes):
```

```
            cells[(d, b)].append(r)
            sem_days[r.sem].append(d)
            sem_blocks[r.sem][d] = b
            # Requested rule: first-semester lab on Wednesday must be 8:00-11:00.
            if r.sem == 1 and wednesday is not None and d == wednesday and b != 0:
                hard += 1
            if any(d in P.unavail.get(k, EMPTY) for k in r.base_keys):
                hard += 1
        # Keep a first-semester lab on Wednesday, specifically in the 8:00-11:00 block.
        wednesday = next((i for i, day in enumerate(P.days) if clean(day).lower().startswith("wed")),
None)
        if wednesday is not None and any(r.sem == 1 for r in self.rounds):
            if wednesday not in sem_days.get(1, []):
                hard += 1
        for cell, rs in cells.items():
            tot = sum(len(r.entries) for r in rs)
            if tot > self.cap:
                hard += tot - self.cap
            seen = set()
            for r in rs:
                if seen & r.base_keys:
                    hard += 1
                seen |= r.base_keys
        per_day, per_block = Counter(), Counter()
        for sem, ds in sem_days.items():
            c = Counter(ds)
            hard += sum(v - 1 for v in c.values() if v > 1)
            u = sorted(c)
            soft += 2 * sum(1 for a, b in zip(u, u[1:]) if b - a == 1)
            for d in u:
                per_day[d] += 1
            for d, b in sem_blocks[sem].items():
```

```
            per_block[b] += 1
        if sem in self.need:
            total, per_day_cap = self.need[sem]
            capacity = 0
            for d in range(self.nd):
                blk = P.block_slots[sem_blocks[sem][d]] if d in sem_blocks[sem] else EMPTY
                capacity += min(per_day_cap, self.ns - len(blk))
            if capacity < total:
                hard += total - capacity
    soft += sum(v * v for v in per_day.values()) + 0.5 * sum(v * v for v in per_block.values())
    return hard * HARD + soft

def run(self, pop_size=80, gens=220):
    rng, cells, n = self.rng, self.cells, len(self.rounds)
    if n == 0:
        return []
    pop = [[rng.choice(cells) for _ in range(n)] for _ in range(pop_size)]
    scored = [(self.cost(g), g) for g in pop]
    best, stale = min(scored, key=lambda x: x[0]), 0
    for _ in range(gens):
        scored.sort(key=lambda x: x[0])
        nxt = [g[:] for _, g in scored[:6]]
        while len(nxt) < pop_size:
            a = min(rng.sample(scored, 3), key=lambda x: x[0])[1]
            b = min(rng.sample(scored, 3), key=lambda x: x[0])[1]
            cut = rng.randrange(1, n) if n > 1 else 0
            child = a[:cut] + b[cut:]
            if rng.random() < 0.5:
                child[rng.randrange(n)] = rng.choice(cells)
            if rng.random() < 0.2 and n > 1:
                i, j = rng.sample(range(n), 2)
                child[i], child[j] = child[j], child[i]
```

```
            nxt.append(child)
        scored = [(self.cost(g), g) for g in nxt]
        cur = min(scored, key=lambda x: x[0])
        if cur[0] < best[0]:
            best, stale = cur, 0
        else:
            stale += 1
        if best[0] < HARD and stale > 40:
            break
    return best[1]


def assign_lab_faculty(P, rounds):
    """Exactly two faculty per batch lab: subject teacher(s) + a lab-qualified CS teacher."""
    load = Counter()
    for sem in P.theory_sems:
        for s in P.theory[sem]:
            for k in s.keys:
                load[k] += s.periods
    sticky = {}
    busy_cell = defaultdict(set)
    for rd in sorted(rounds, key=lambda r: (r.day, r.block, -r.sem)):
        busy = busy_cell[(rd.day, rd.block)]
        reserved = set(rd.base_keys)        # every subject teacher of this block is spoken for
        for e in rd.entries:
            team = list(e["base"])[:2]
            e["lab_room"] = P.labs[e["bi"] % len(P.labs)] if P.labs else "Lab"
            while len(team) < 2:
                sub = e["sub"]
                taken = set(team) | busy | reserved
                cands = [k for k in P.pool
                         if k not in taken and rd.day not in P.unavail.get(k, EMPTY)]
                if not cands:
```

```
                break
            st = sticky.get((rd.sem, sub.name))
            pick = st if st in cands else min(cands, key=lambda k: (load[k], P.disp.get(k, k)))
            sticky[(rd.sem, sub.name)] = pick
            load[pick] += 3
            team.append(pick)
        busy.update(team)
        e["team"] = team
  lab_busy = defaultdict(set)
  for rd in rounds:
    keys = {k for e in rd.entries for k in e["team"]}
    for sl in P.block_slots[rd.block]:
      lab_busy[(rd.day, sl)] |= keys
  return {k: frozenset(v) for k, v in lab_busy.items()}
# ============================================================================
# 4. GA - THEORY PLACEMENT
# ============================================================================
class TheoryGA:
  """
  Chromosome : ind[sem][subject_index] = [(day, slot), ...]   (one entry per weekly period)
  Hard       : slot clash inside a semester, theory inside own lab, same subject twice a day,
               faculty clash (theory+theory, theory+lab), faculty unavailable, no free room
  Soft       : gaps inside a day, needlessly late starts, uneven daily load (lab days lighter), subjects on
               consecutive days
  Fitness    : 1 / (1 + conflict score)
  """
  def __init__(self, P, rounds, lab_busy, rng):
    self.P, self.rng, self.lab_busy = P, rng, lab_busy
    self.nd, self.ns = len(P.days), len(P.theory_slots)
    self.sems = P.theory_sems
    self.blocked = {s: [set() for _ in range(self.nd)] for s in self.sems}
```

```
        for r in rounds:
            if r.sem in self.blocked:
                self.blocked[r.sem][r.day] |= P.block_slots[r.block]
        self.free = {s: [[x for x in range(self.ns) if x not in self.blocked[s][d]]
                         for d in range(self.nd)] for s in self.sems}
        self.subs = {s: P.theory[s] for s in self.sems}
        self.targets = {}
        for s in self.sems:
            tot = sum(x.periods for x in self.subs[s])
            w = [0.6 if self.blocked[s][d] else 1.0 for d in range(self.nd)]
            tw = sum(w)
            self.targets[s] = [tot * x / tw for x in w]
        self.place = [(s, ti, i) for s in self.sems for ti, sub in enumerate(self.subs[s])
                      for i in range(sub.periods)]
        # Requested first-semester Wednesday arrangement: exactly one theory class
        # at 11:50-12:40, with no other first-semester theory periods that day.
        self.wednesday = next((i for i, day in enumerate(P.days)
                               if clean(day).lower().startswith("wed")), None)
        self.wed_target = next((i for i, slot in enumerate(P.theory_slots)
                                if parse_range(slot)[0] == parse_clock("11:50")), None)

    # ---- construction ----------------------------------------------------
    def random_ind(self):
        rng, ind = self.rng, {}
        for s in self.sems:
            used = set()
            ind[s] = [None] * len(self.subs[s])
            order = sorted(range(len(self.subs[s])), key=lambda i: (-self.subs[s][i].periods,
rng.random()))
            for ti in order:
                sub, pl, cnt = self.subs[s][ti], [], Counter()
                days = list(range(self.nd))
```

```
            rng.shuffle(days)
            for _ in range(sub.periods):
                cand = [d for d in days if cnt[d] < sub.maxpd and
                        any((d, x) not in used for x in self.free[s][d])]
                if not cand:
                    cand = days
                d = cand[0] if len(cand) == 1 else rng.choice(cand)
                opts = [x for x in self.free[s][d] if (d, x) not in used] or list(range(self.ns))
                sl = rng.choice(opts)
                used.add((d, sl))
                cnt[d] += 1
                pl.append((d, sl))
            ind[s][ti] = pl
    return ind

# ---- fitness ------------------------------------------------------
def evaluate(self, ind, rec=None):
    P, hard, soft, bad = self.P, 0, 0.0, []
    fac_used, room_used = {}, {}
    unavail, lab_busy = P.unavail, self.lab_busy
    for s in self.sems:                          # high priority first
        taken, per_day = set(), defaultdict(list)
        blocked = self.blocked[s]
        rooms = P.rooms_for[s]
        for ti, sub in enumerate(self.subs[s]):
            cnt = Counter()
            keys = sub.keys
            for i, (d, sl) in enumerate(ind[s][ti]):
                ref = (s, ti, i)
                if (d, sl) in taken:
                    hard += 1
                    bad.append(ref)
```

```
# Keep the first-semester Wednesday class at 11:50-12:40.
if s == 1 and self.wednesday is not None and d == self.wednesday:
    if self.wed_target is None or sl != self.wed_target:
        hard += 1
        bad.append(ref)
taken.add((d, sl))
if sl in blocked[d]:
    hard += 1
    bad.append(ref)
cnt[d] += 1
if cnt[d] > sub.maxpd:
    hard += 1
    bad.append(ref)
per_day[d].append(sl)
if keys:
    fu = fac_used.get((d, sl))
    if fu is None:
        fu = fac_used[(d, sl)] = set()
    if (fu & keys) or (keys & lab_busy.get((d, sl), EMPTY)):
        hard += 1
        bad.append(ref)
    if unavail and any(d in unavail.get(k, EMPTY) for k in keys):
        hard += 1
        bad.append(ref)
    fu |= keys
ru = room_used.get((d, sl))
if ru is None:
    ru = room_used[(d, sl)] = set()
chosen = None
for rm in rooms:
    if rm not in ru:
        chosen = rm
```

```
                    break
            if chosen is None:
                hard += 1
                bad.append(ref)
            else:
                ru.add(chosen)
                if rec is not None:
                    rec[ref] = chosen
        # Require one first-semester theory class on Wednesday at 11:50-12:40.
        if s == 1 and self.wednesday is not None and self.wed_target is not None:
            if not any(d == self.wednesday and sl == self.wed_target
                       for ti, sub in enumerate(self.subs[s])
                       for d, sl in ind[s][ti]):
                hard += 1
        # soft terms
        tg = self.targets[s]
        for d in range(self.nd):
            sl_list = per_day.get(d, [])
            soft += 2.0 * (len(sl_list) - tg[d]) ** 2
            if sl_list:
                lo, hi = min(sl_list), max(sl_list)
                occ = set(sl_list) | blocked[d]
                soft += 4.0 * sum(1 for x in range(lo, hi + 1) if x not in occ)
                soft += 0.6 * lo          # mild tie-breaker: avoid needlessly late starts
        for ti, sub in enumerate(self.subs[s]):
            ds = sorted({d for d, _ in ind[s][ti]})
            soft += 1.5 * sum(1 for a, b in zip(ds, ds[1:]) if b - a == 1)
    return hard, soft, bad

# ---- operators ----------------------------------------------------------
def mutate(self, ind, bad):
    rng = self.rng
```

```
        child = {s: [list(p) for p in ind[s]] for s in self.sems}
        for _ in range(1 + (rng.random() < 0.5)):
            ref = rng.choice(bad) if bad and rng.random() < 0.75 else rng.choice(self.place)
            s, ti, i = ref
            sub = self.subs[s][ti]
            pls = child[s][ti]
            if rng.random() < 0.65:                    # move
                used = {x for p in child[s] for x in p}
                have = {d for k, (d, _) in enumerate(pls) if k != i}
                days = [d for d in range(self.nd) if d not in have] or list(range(self.nd))
                d = rng.choice(days)
                opts = [x for x in self.free[s][d] if (d, x) not in used] or self.free[s][d] or list(range(self.ns))
                pls[i] = (d, rng.choice(opts))
            else:                                      # swap two periods of the semester
                ti2 = rng.randrange(len(self.subs[s]))
                j = rng.randrange(len(child[s][ti2]))
                pls[i], child[s][ti2][j] = child[s][ti2][j], pls[i]
        return child

    def crossover(self, a, b):
        rng, child = self.rng, {}
        for s in self.sems:
            if rng.random() < 0.3:                     # mix subject by subject
                child[s] = [list(a[s][ti] if rng.random() < 0.5 else b[s][ti])
                            for ti in range(len(self.subs[s]))]
            else:
                src = a if rng.random() < 0.5 else b
                child[s] = [list(p) for p in src[s]]
        return child

    def run(self, pop_size=120, max_gens=700, time_cap=25.0, patience=70):
        rng, t0 = self.rng, time.time()
```

```
pop = []
for _ in range(pop_size):
    ind = self.random_ind()
    h, sft, bad = self.evaluate(ind)
    pop.append((h * HARD + sft, h, sft, bad, ind))
best, stale, gen = min(pop, key=lambda x: x[0]), 0, 0
for gen in range(1, max_gens + 1):
    pop.sort(key=lambda x: x[0])
    nxt = pop[:8]
    while len(nxt) < pop_size:
        a = min(rng.sample(pop, 4), key=lambda x: x[0])
        b = min(rng.sample(pop, 4), key=lambda x: x[0])
        child = self.crossover(a[4], b[4]) if rng.random() < 0.85 else \
            {s: [list(p) for p in a[4][s]] for s in self.sems}
        child = self.mutate(child, a[3] if rng.random() < 0.5 else b[3])
        h, sft, bad = self.evaluate(child)
        nxt.append((h * HARD + sft, h, sft, bad, child))
    pop = nxt
    cur = min(pop, key=lambda x: x[0])
    if cur[0] < best[0] - 1e-9:
        best, stale = cur, 0
    else:
        stale += 1
    if best[1] == 0 and stale >= patience:
        break
    if time.time() - t0 > time_cap and best[1] == 0:
        break
    if time.time() - t0 > time_cap * 2:
        break
return best[4], dict(generations=gen, hard=best[1], soft=round(best[2], 1),
                     fitness=1.0 / (1.0 + best[0]), seconds=round(time.time() - t0, 1))
```

```
    def build(self, ind):
        rec = {}
        self.evaluate(ind, rec)
        table = {s: [[None] * self.ns for _ in range(self.nd)] for s in self.sems}
        for s in self.sems:
            for ti, sub in enumerate(self.subs[s]):
                for i, (d, sl) in enumerate(ind[s][ti]):
                    table[s][d][sl] = dict(
                        subject=sub.name,
                        faculty=" & ".join(sub.teachers) if sub.teachers else "TBA",
                        room=rec.get((s, ti, i), "?"), sub=sub)
        return table
# ============================================================================
# 5. INDEPENDENT VALIDATION
# ============================================================================
def validate(P, theory, rounds):
    """Re-checks the finished timetable from scratch. Returns [(check, passed, detail)]."""
    nd, ns = len(P.days), len(P.theory_slots)
    out = []

    def add(name, errs):
        out.append((name, not errs, "; ".join(errs[:3]) + (" ..." if len(errs) > 3 else "")))

    # weekly counts
    errs = []
    for s in P.theory_sems:
        cnt = Counter(e["subject"] for d in range(nd) for e in theory[s][d] if e)
        for sub in P.theory[s]:
            if cnt[sub.name] != sub.periods:
                errs.append(f"Sem {s} {sub.name}: {cnt[sub.name]} of {sub.periods}")
    add("Weekly theory periods match the dataset", errs)
```

```
# repeat per day
errs = []
for s in P.theory_sems:
    for d in range(nd):
        c = Counter(e["subject"] for e in theory[s][d] if e)
        for sub in P.theory[s]:
            if c[sub.name] > sub.maxpd:
                errs.append(f"Sem {s} {sub.name} repeats on {P.days[d]}")
add("No subject repeated on the same day", errs)

# own lab overlap
errs = []
for r in rounds:
    for sl in P.block_slots[r.block]:
        if r.sem in theory and theory[r.sem][r.day][sl]:
            errs.append(f"Sem {r.sem} theory during its lab on {P.days[r.day]}")
add("No theory during the semester's own lab", errs)

# faculty clashes: theory+theory (same period), lab+lab (same block), theory+lab (overlap)
occ = defaultdict(list)                    # (day, period) -> teacher keys in theory
for s in P.theory_sems:
    for d in range(nd):
        for sl in range(ns):
            e = theory[s][d][sl]
            if e:
                occ[(d, sl)].extend(e["sub"].keys)
lab_cell = defaultdict(list)               # (day, block) -> teacher keys in labs
for r in rounds:
    for e in r.entries:
        lab_cell[(r.day, r.block)].extend(e["team"])
errs = []
for (d, sl), lst in occ.items():
```

```
    for k, n in Counter(lst).items():
        if n > 1:
            errs.append(f"{P.disp.get(k, k)} double-booked {P.days[d]} {P.theory_slots[sl]}")
for (d, b), lst in lab_cell.items():
    for k, n in Counter(lst).items():
        if n > 1:
            errs.append(f"{P.disp.get(k, k)} double-booked in labs {P.days[d]} {P.lab_slots[b]}")
    for sl in P.block_slots[b]:
        for k in set(lst) & set(occ.get((d, sl), [])):
            errs.append(f"{P.disp.get(k, k)} in a lab and a class {P.days[d]} {P.theory_slots[sl]}")
add("No faculty clash (theory or lab)", errs)

# unavailable
errs = []
for (d, sl), lst in occ.items():
    for k in lst:
        if d in P.unavail.get(k, EMPTY):
            errs.append(f"{P.disp.get(k, k)} unavailable {P.days[d]}")
for (d, b), lst in lab_cell.items():
    for k in lst:
        if d in P.unavail.get(k, EMPTY):
            errs.append(f"{P.disp.get(k, k)} unavailable {P.days[d]} (lab)")
add("Faculty availability respected", errs)

# rooms
errs, errs2 = [], []
for d in range(nd):
    for sl in range(ns):
        seen = set()
        for s in P.theory_sems:
            e = theory[s][d][sl]
            if not e:
```

```
            continue
        if e["room"] in seen or e["room"] == "?":
            errs.append(f"Room clash {e['room']} {P.days[d]} {P.theory_slots[sl]}")
        seen.add(e["room"])
        if e["room"] not in P.rooms_for[s]:
            errs2.append(f"Sem {s} in {e['room']}")
add("No classroom clash", errs)
add("Only permitted classrooms used", errs2)

# labs
errs = []
cells = defaultdict(list)
for r in rounds:
    cells[(r.day, r.block)].append(r)
    used = [e["lab_room"] for e in r.entries]
    if len(set(used)) != len(used):
        errs.append(f"Lab room reused inside Sem {r.sem} block")
for (d, b), rs in cells.items():
    used = [e["lab_room"] for r in rs for e in r.entries]
    if len(used) != len(set(used)):
        errs.append(f"Laboratory clash {P.days[d]} {P.lab_slots[b]}")
add("No laboratory clash", errs)

errs = []
per = defaultdict(Counter)
for r in rounds:
    per[r.sem][r.day] += 1
for s, c in per.items():
    for d, n in c.items():
        if n > 1:
            errs.append(f"Sem {s} has {n} lab blocks on {P.days[d]}")
add("At most one lab block per semester per day", errs)
```

```
    errs = []
    done = defaultdict(Counter)
    for r in rounds:
        for e in r.entries:
            done[(r.sem, e["batch"])][e["sub"].name] += 1
    for s in P.sems:
        for sub in P.labsubs.get(s, []):
            for b in P.batches[s]:
                if done[(s, b)][sub.name] != sub.sessions:
                    errs.append(f"Sem {s} {b} {sub.name}: {done[(s, b)][sub.name]} of {sub.sessions}")
    add("Every batch gets every required lab session", errs)

    errs = []
    for r in rounds:
        for e in r.entries:
            if len(e["team"]) != 2:
                errs.append(f"Sem {r.sem} {e['sub'].name} {e['batch']} has {len(e['team'])} faculty")
    add("Exactly two faculty per lab", errs)

    errs = []
    for r in rounds:
        for e in r.entries:
            for k in e["team"]:
                if k in P.language_keys and k not in e["base"]:
                    errs.append(f"Language faculty {P.disp.get(k, k)} in a lab")
    add("Language faculty never assigned to labs", errs)
    return out
# ============================================================================
# 6. SOLVE
# ============================================================================
def solve(P, seed=42, attempts=6, log=print):
```

```
    t0 = time.time()
    last = None
    for attempt in range(attempts):
        rng = random.Random(seed + attempt * 977)
        rounds = build_rounds(P)
        placer = LabPlacer(P, rounds, rng)
        genes = placer.run()
        for r, (d, b) in zip(rounds, genes):
            r.day, r.block = d, b
        lab_busy = assign_lab_faculty(P, rounds)
        ga = TheoryGA(P, rounds, lab_busy, rng)
        ind, info = ga.run() if P.theory_sems else ({}, dict(generations=0, hard=0, soft=0, fitness=1,
seconds=0))
        theory = ga.build(ind) if P.theory_sems else {}
        checks = validate(P, theory, rounds)
        failed = sum(1 for _, ok, _ in checks if not ok)
        log(f"  attempt {attempt + 1}: GA generations={info['generations']}, "
            f"hard conflicts={info['hard']}, validation failures={failed}")
        last = dict(rounds=rounds, theory=theory, checks=checks, info=info, attempt=attempt + 1)
        if failed == 0:
            break
    last["seconds"] = round(time.time() - t0, 1)
    last["passed"] = sum(1 for _, ok, _ in last["checks"] if ok)
    last["failed"] = len(last["checks"]) - last["passed"]
    return last
# ============================================================================
# 7. HTML FRAGMENTS (existing classes only - no CSS is added or changed)
# ============================================================================
def esc(v):
    return html.escape(str(v), quote=True)
def theory_card(e):
    if not e:
```

```
        return '<span class="slot-chip free">—</span>'
    return ('<span class="slot-chip theory">' + esc(e["subject"]) +
            f'<em>{esc(e["room"])} · {esc(e["faculty"])}</em></span>')
def lab_cards(P, r):
    out = []
    for e in r.entries:
        team = " & ".join(P.disp.get(k, k) for k in e["team"])
        out.append('<span class="slot-chip lab">' + esc(e["sub"].lab_name) +
                   f'<em>{esc(e["batch"])} · {esc(e["lab_room"])} · {esc(team)}</em>'
                   f'<small>{esc(pretty_range(P.lab_slots[r.block]))}</small></span>')
    return "".join(out)

def sem_table(P, res, sem):
    theory, rounds = res["theory"], res["rounds"]

    # Interleave lab blocks with theory periods by their real start times.
    events = []
    for sl, slot in enumerate(P.theory_slots):
        st, en = P.theory_rng[sl]
        events.append((st, 1, "theory", sl, slot))
    for bi, slot in enumerate(P.lab_slots):
        st, en = P.lab_rng[bi]
        events.append((st, 0, "lab", bi, slot))
    events.sort(key=lambda x: (x[0], x[1]))

    rows = []
    for d, dn in enumerate(P.days):
        cells = [f'<td class="tt-time">{esc(dn)}</td>']
        for _, _, kind, idx, slot in events:
            if kind == "lab":
                matching = [rr for rr in rounds
                            if rr.sem == sem and rr.day == d and rr.block == idx]
```

```
            chips = "".join(lab_cards(P, rr) for rr in matching)
            if not chips:
                chips = '<span class="slot-chip free">—</span>'
            cells.append(f"<td>{chips}</td>")
        else:
            e = theory[sem][d][idx] if sem in theory else None
            cells.append(f"<td>{theory_card(e)}</td>")
    rows.append("<tr>" + "".join(cells) + "</tr>")

head = ['<th class="tt-time">Day</th>'] + [
    f"<th>{esc(pretty_range(slot))}</th>" for _, _, _, _, slot in events
]
return (f'<div class="grid-wrap sem-grid" id="sem{sem}"><table class="tt-grid"><thead><tr>'
        + "".join(head) + "</tr></thead><tbody>" + "".join(rows) + "</tbody></table></div>")

def timetable_inner(P, res, live=False):
    sems = sorted(P.sems)
    ok = res["failed"] == 0
    verdict = (f'All {res["passed"]} hard-constraint checks passed - no faculty, room or laboratory clashes."
               if ok else f'{res["failed"]} of {len(res["checks"])} checks failed - see the note below the grid.")
    tabs = "".join(f'<button class="tab{" is-active" if i == 0 else ""}" data-sem="sem{s}">Semester {s}</button>'
                   for i, s in enumerate(sems))
    grids = "".join(sem_table(P, res, s).replace('class="grid-wrap sem-grid"',
                                                 'class="grid-wrap sem-grid is-active"' if s == sems[0] else
                                                 'class="grid-wrap sem-grid"') for s in sems)
    info = res["info"]
    fine = (f'Generated from the uploaded dataset by a Genetic Algorithm '
            f'(lab placement + theory placement; {info["generations"]} generations, '
            f'fitness {info["fitness"]:.4f}, {res["seconds"]}s).")
```

```
    if not ok:
        fine += " Failed: " + " | ".join(f"{n}: {d}" for n, okc, d in res["checks"] if not okc)
    return (f'<header class="view-head"><p class="eyebrow">Step 2</p><h1>Generated timetable</h1>'
            f'<p class="lede">Theory in amber, labs in teal. {esc(verdict)}</p></header>'
            + master_html(P, res) +
            f'<div class="panel-head" style="margin-top:28px"><h2>Semester-wise view</h2></div>'
            f'<div class="tabs" role="tablist">{tabs}</div>{grids}'
            f'<p class="fineprint">{esc(fine)}</p>'
            + ('<p class="fineprint"><a class="btn btn-quiet" href="/api/timetable.xlsx" '
               'download="timetable.xlsx">Download master &amp;amp; semester timetables (.xlsx)</a></p>' if live else "")
    )
def fragments(P, res):
    n_fac = len(set(P.teaches) | P.faculty_sheet_keys)
    years = len(P.sems)
    stats = (
        f'<div class="stat"><span class="stat-value">{P.n_scheduled_rows:02d}</span>'
        f'<span class="stat-label">Subjects mapped</span><span class="stat-sub">across {years} semesters</span></div>'
        f'<div class="stat"><span class="stat-value">{n_fac:02d}</span>'
        f'<span class="stat-label">Faculty on record</span><span class="stat-sub">availability captured</span></div>'
        f'<div class="stat"><span class="stat-value">{len(P.classrooms) + len(P.labs):02d}</span>'
        f'<span class="stat-label">Rooms ready</span><span class="stat-sub">{len(P.classrooms)} classrooms · '
        f'{len(P.labs)} laboratories</span></div>'
        f'<div class="stat stat-flag"><span class="stat-value">{P.flag_rows:02d}</span>'
        f'<span class="stat-label">Rows to review</span><span class="stat-sub">see the data checks</span></div>')

    if P.notes:
```

```
    checks = "".join(f"<li><strong>{esc(t)}</strong><span>{esc(x)}</span></li>" for t, x in P.notes)
  else:
    checks = "<li><strong>No issues found</strong><span>The dataset passed every check.</span></li>"
  low = sorted(P.sems)[0]
  items = []
  if low in res["theory"]:
    for sl, e in enumerate(res["theory"][low][0]):
      if e:
        h, m = divmod(P.theory_rng[sl][0], 60)
        items.append((P.theory_rng[sl][0], f"{h:02d}:{m:02d}", theory_card(e)))
  for r in res["rounds"]:
    if r.sem == low and r.day == 0:
      h, m = divmod(P.lab_rng[r.block][0], 60)
      items.append((P.lab_rng[r.block][0], f"{h:02d}:{m:02d}", lab_cards(P, r)))
  items.sort(key=lambda x: x[0])
  glimpse = "".join(f'<li><span class="glimpse-time">{t}</span>{c}</li>' for _, t, c in items) or \
    '<li><span class="glimpse-time">—</span><span class="slot-chip free">No classes</span></li>'

  rows = []
  empty_tba = '<span class="cell-empty">TBA</span>'
  empty_missing = '<span class="cell-empty">missing</span>'
  for r in P.table_rows:
    faculty_cell = esc(r["faculty"]) or empty_tba
    marks_cell = esc(r["marks"]) or empty_missing
    cls = ' class="is-flagged"' if r["flagged"] else ""

rows.append(f'<tr{cls}><td>{r["sem"]}</td><td>{esc(r["year"])}</td><td>{esc(r["subject"])}</td>'
                f'<td>{faculty_cell}</td>'
```

```
            f'<td>{esc(r["type"])}</td><td>{marks_cell}</td>'
            f'<td>{r["theory"]}</td><td>{esc(r["lab"])}</td></tr>')

    rooms = "".join(f"<tr><td>{esc(n)}</td><td>{esc(t)}</td><td>{c}</td><td>{esc(u)}</td></tr>"
                    for n, t, c, u in P.room_info)
    by_t = defaultdict(lambda: [[], set()])
    for k, lst in P.teaches.items():
        for subj, sem in lst:
            if subj not in by_t[k][0]:
                by_t[k][0].append(subj)
            by_t[k][1].add((sem + 1) // 2)
    fac = "".join(f"<tr><td>{esc(P.disp.get(k, k))}</td><td>{esc(' · '.join(v[0]))}</td>"
                  f"<td>{esc(', '.join(ordinal(y) for y in sorted(v[1])))}</td></tr>"
                  for k, v in sorted(by_t.items(), key=lambda kv: P.disp.get(kv[0], kv[0]).lower()))

    return dict(ok=True, stats=stats, checks=checks, flagged=len(P.notes), glimpse=glimpse,
                glimpse_tag=f"Semester {low}", rows=len(P.table_rows), dataset_rows="".join(rows),
                timetable=timetable_inner(P, res, True), rooms=rooms, faculty=fac,
                clean=res["failed"] == 0, passed=res["passed"], failed=res["failed"],
                seconds=res["seconds"],
                checks_detail=[dict(name=n, ok=o, detail=d) for n, o, d in res["checks"]])

def generate_from(source, filename="dataset.xlsx", seed=42, log=print):
    sheets = read_sheets(source, filename)
    P = load_problem(sheets)
    log(f"  semesters {sorted(P.sems)} | classrooms {P.classrooms} | labs {P.labs}")
    res = solve(P, seed=seed, log=log)
    return P, res
# ============================================================================
# 7b. MASTER TIMETABLE + SEMESTER TIMETABLES  (page and Excel share the same data)
# ============================================================================
```

```
def sem_label(sem):
    return f"{ordinal(sem)} Sem"


def build_master(P, res):
    """[(day, [(sem, [cell,...]), ...]), ...]  one row per semester per day, 7 theory columns.
    A lab becomes one merged cell spanning the theory periods it occupies."""
    ns = len(P.theory_slots)
    out = []
    for d, dn in enumerate(P.days):
        rows = []
        for sem in sorted(P.sems):
            lab = next((r for r in res["rounds"] if r.sem == sem and r.day == d), None)
            blocked = P.block_slots[lab.block] if lab else frozenset()
            cells, sl = [], 0
            while sl < ns:
                if sl in blocked:
                    c1 = sl
                    while c1 + 1 < ns and (c1 + 1) in blocked:
                        c1 += 1
                    cells.append(dict(kind="lab", c0=sl, c1=c1, round=lab))
                    sl = c1 + 1
                else:
                    e = res["theory"][sem][d][sl] if sem in res["theory"] else None
                    cells.append(dict(kind="theory" if e else "free", c0=sl, c1=sl, e=e))
                    sl += 1
            rows.append((sem, cells))
        out.append((dn, rows))
    return out


def master_text(P, c):
    if c["kind"] == "theory":
        return f"{c['e']['subject']} – {c['e']['room']}\n{c['e']['faculty']}"
```

```
    if c["kind"] == "lab":
        r = c["round"]
        lines = [f"LAB ({pretty_range(P.lab_slots[r.block])})"]
        for e in r.entries:
            team = " & ".join(P.disp.get(k, k) for k in e["team"])
            lines.append(f"B{e['bi'] + 1}: {e['sub'].lab_name} ({e['lab_room']}) – {team}")
        return "\n".join(lines)
    return ""

def master_html(P, res):
    head = ('<th class="tt-time">DAYS</th><th class="tt-time">SEMESTERS</th>'
            + "".join(f"<th>{esc(pretty_range(x))}</th>" for x in P.theory_slots))
    body = []
    for dn, rows in build_master(P, res):
        for i, (sem, cells) in enumerate(rows):
            tds = ""
            if i == 0:
                tds += f'<td class="tt-time" rowspan="{len(rows)}"><strong>{esc(dn.upper())}</strong></td>'
            tds += f'<td class="tt-time">{esc(sem_label(sem))}</td>'
            for c in cells:
                span = c["c1"] - c["c0"] + 1
                if c["kind"] == "lab":
                    tds += f'<td colspan="{span}">{lab_cards(P, c["round"])}</td>'
                elif c["kind"] == "theory":
                    tds += f"<td>{theory_card(c['e'])}</td>"
                else:
                    tds += '<td><span class="slot-chip free">—</span></td>'
            body.append(f"<tr>{tds}</tr>")
    return ('<div class="panel-head"><h2>Master timetable</h2>'
            '<span class="panel-tag">all semesters</span></div>'
            f'<div class="grid-wrap"><table class="tt-grid"><thead><tr>{head}</tr></thead>'
```

```
        f'<tbody>{"".join(body)}</tbody></table></div>')

def sem_text(P, e):
    return f"{e['subject']} – {e['room']}\n{e['faculty']}" if e else None

def sem_lab_text(P, r):
    lines = [f"LAB ({pretty_range(P.lab_slots[r.block])})"]
    for e in r.entries:
        team = " & ".join(P.disp.get(k, k) for k in e["team"])
        lines.append(f"B{e['bi'] + 1}: {e['sub'].lab_name} ({e['lab_room']}) – {team}")
    return "\n".join(lines)

def write_timetable_xlsx(P, res, target):
    """Workbook: 'Master Timetable' + one sheet per semester. target: path or file-like."""
    from openpyxl import Workbook
    from openpyxl.styles import Alignment, Border, Font, Side
    wb = Workbook()
    thin = Side(style="thin")
    box = Border(left=thin, right=thin, top=thin, bottom=thin)
    FONT = "Aptos Narrow"
    ns = len(P.theory_slots)

    def style(ws, nrows, ncols, first_cols):
        for row in ws.iter_rows(min_row=1, max_row=nrows, max_col=ncols):
            for c in row:
                c.border = box
                c.font = Font(name=FONT, size=11, bold=c.row == 1 or c.column <= first_cols or
                              c.value is not None)
                c.alignment = Alignment(horizontal="center" if (c.row == 1 or c.column == 1) else None,
                                        vertical="center", wrap_text=True)

    # ---- master ------------------------------------------------------
```

```
    ms = wb.active
    ms.title = "Master Timetable"
    ms.append(["DAYS", "SEMESTERS"] + [pretty_range(x) for x in P.theory_slots])
    r0 = 2
    for dn, rows in build_master(P, res):
        for sem, cells in rows:
            ms.cell(row=r0, column=2, value=sem_label(sem))
            for c in cells:
                ms.cell(row=r0, column=c["c0"] + 3, value=master_text(P, c) or None)
            r0 += 1
    ncols = ns + 2
    style(ms, r0 - 1, ncols, 2)
    r = 2
    for dn, rows in build_master(P, res):
        ms.cell(row=r, column=1, value=dn.upper())
        ms.merge_cells(start_row=r, start_column=1, end_row=r + len(rows) - 1, end_column=1)
        for sem, cells in rows:
            for c in cells:
                if c["c1"] > c["c0"]:
                    ms.merge_cells(start_row=r,       start_column=c["c0"]       +       3,       end_row=r,
end_column=c["c1"] + 3)
            narrow_lab = any(c["kind"] == "lab" and c["c1"] == c["c0"] for c in cells)
            ms.row_dimensions[r].height = 110 if narrow_lab else 62
            r += 1
    ms.row_dimensions[1].height = 28.8
    ms.column_dimensions["A"].width = 17
    ms.column_dimensions["B"].width = 15
    for ci in range(3, ncols + 1):
        ms.column_dimensions[chr(64 + ci)].width = 27
    ms.column_dimensions[chr(64 + ncols)].width = 44


    # ---- one sheet per semester (Day x periods + Lab Block) -------------------
```

```
    for sem in sorted(P.sems):
        ws = wb.create_sheet(sem_label(sem))
        ws.append(["Day"] + [pretty_range(x) for x in P.theory_slots] + ["Lab Block"])
        for d, dn in enumerate(P.days):
            row = [dn]
            for sl in range(ns):
                e = res["theory"][sem][d][sl] if sem in res["theory"] else None
                row.append(sem_text(P, e))
            lab = next((x for x in res["rounds"] if x.sem == sem and x.day == d), None)
            row.append(sem_lab_text(P, lab) if lab else "—")
            ws.append(row)
        style(ws, len(P.days) + 1, ns + 2, 1)
        ws.row_dimensions[1].height = 28.8
        for ri in range(2, len(P.days) + 2):
            ws.row_dimensions[ri].height = 62
        ws.column_dimensions["A"].width = 14
        for ci in range(2, ns + 2):
            ws.column_dimensions[chr(64 + ci)].width = 24
        ws.column_dimensions[chr(64 + ns + 2)].width = 44
    wb.save(target)
# ============================================================================
# 8. LIVE PAGE: served index.html + injected drag-and-drop glue
# ============================================================================
INJECT = r"""
<script id="tt-live">
(function () {
  const section = document.getElementById('timetable');
  if (!section) return;

  function setStatus(msg, bad) {
    const el = document.getElementById('ttStatus');
    if (!el) return;
```

```
  el.textContent = msg;
  el.style.color = bad ? 'var(--warn, #b00020)' : '';
 }

 async function generate(file) {
  if (!file) return;
  if (!/\.(xlsx|xlsm|xls)$/i.test(file.name)) {
   setStatus('Please choose the Excel workbook (.xlsx).', true);
   return;
  }
  setStatus('Generating timetable from ' + file.name + ' ...', false);
  try {
   const res = await fetch('/api/generate', {
    method: 'POST',
    headers: { 'X-Filename': encodeURIComponent(file.name) },
    body: file
   });
   const f = await res.json();
   if (!f.ok) throw new Error(f.error || 'Generation failed');
   section.innerHTML = f.timetable;
   setStatus('', false);
  } catch (err) {
   setStatus('Could not generate: ' + (err.message || err), true);
  }
 }

 // Drag and drop: the page must not open a dropped file, only the drop zone takes it
 document.addEventListener('dragover', e => {
  e.preventDefault();
  const z = e.target.closest && e.target.closest('.tt-drop');
  document.querySelectorAll('.tt-drop.is-dragging').forEach(x => { if (x !== z)
x.classList.remove('is-dragging'); });
```

```
    if (z) z.classList.add('is-dragging');
  });
  document.addEventListener('dragleave', e => {
    const z = e.target.closest && e.target.closest('.tt-drop');
    if (z && !z.contains(e.relatedTarget)) z.classList.remove('is-dragging');
  });
  document.addEventListener('drop', e => {
    e.preventDefault();
    document.querySelectorAll('.tt-drop.is-dragging').forEach(x => x.classList.remove('is-
dragging'));
    const z = e.target.closest && e.target.closest('.tt-drop');
    if (z && e.dataTransfer && e.dataTransfer.files.length) {
      generate(e.dataTransfer.files[0]);
    }
  });

  // Upload buttons (first upload on the page and re-upload after generating)
  document.addEventListener('change', e => {
    if (e.target && (e.target.id === 'ttFile' || e.target.id === 'ttFileAgain')) {
      generate(e.target.files[0]);
      e.target.value = '';
    }
  });

  // Semester tabs inside the generated timetable
  section.addEventListener('click', e => {
    const t = e.target.closest('.tab');
    if (!t) return;
    section.querySelectorAll('.tab').forEach(x => x.classList.remove('is-active'));
    section.querySelectorAll('.sem-grid').forEach(x => x.classList.remove('is-active'));
    t.classList.add('is-active');
    const g = document.getElementById(t.dataset.sem);
```

```
    if (g) g.classList.add('is-active');
  });
})();
</script>
"""
_lock = threading.Lock()
LAST = {}
def find_existing(names):
    for n in names:
        p = BASE_DIR / n
        if p.exists():
            return p
    return None

class Handler(BaseHTTPRequestHandler):
    def log_message(self, *a):
        pass

    def _send(self, code, body, ctype):
        if isinstance(body, str):
            body = body.encode("utf-8")
        self.send_response(code)
        self.send_header("Content-Type", ctype)
        self.send_header("Content-Length", str(len(body)))
        self.send_header("Cache-Control", "no-store")
        self.end_headers()
        self.wfile.write(body)

    def do_GET(self):
        path = urlparse(self.path).path
        if path in ("/", "/index.html"):
            f = find_existing(HTML_NAMES)
```

```
            if not f:
                return self._send(404, "mLAC Smart Timetable.html not found next to the script",
"text/plain")
            text = f.read_text(encoding="utf-8")
            if 'id="p-generate"' not in text:          # the mLAC site has its own built-in glue
                text = text.replace("</body>", INJECT + "\n</body>", 1) if "</body>" in text else text +
INJECT
            return self._send(200, text, "text/html; charset=utf-8")
        if path == "/api/timetable.xlsx":
            if "P" not in LAST:
                return self._send(404, "Generate a timetable first.", "text/plain")
            buf = io.BytesIO()
            with _lock:
                write_timetable_xlsx(LAST["P"], LAST["res"], buf)
            return self._send(200, buf.getvalue(),
                              "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet")
        if path == "/styles.css":
            f = find_existing(CSS_NAMES)
            if f:
                return self._send(200, f.read_bytes(), "text/css; charset=utf-8")
        ext = Path(path).suffix.lower()
        mimes = {".png": "image/png", ".jpg": "image/jpeg", ".jpeg": "image/jpeg",
                 ".gif": "image/gif", ".svg": "image/svg+xml", ".webp": "image/webp",
                 ".ico": "image/x-icon", ".css": "text/css; charset=utf-8", ".woff2": "font/woff2"}
        if ext in mimes:                              # logo.png, college.jpg, etc.
            f = (BASE_DIR / unquote(path).lstrip("/")).resolve()
            if f.is_file() and BASE_DIR.resolve() in f.parents:
                return self._send(200, f.read_bytes(), mimes[ext])
        self._send(404, "Not found", "text/plain")

    def do_POST(self):
        u = urlparse(self.path)
```

```
        if u.path != "/api/generate":
            return self._send(404, "Not found", "text/plain")
        n = int(self.headers.get("Content-Length") or 0)
        body = self.rfile.read(n) if n else b""
        name = unquote(self.headers.get("X-Filename", "dataset.xlsx"))
        print(f"Received dataset: {name} ({len(body)} bytes) - generating...", flush=True)
        try:
            with _lock:
                if not body:
                    sample = find_existing(SAMPLE_NAMES)
                    if not sample:
                        raise ValueError("No sample dataset next to the script. Drop your own workbook.")
                    P, res = generate_from(sample)
                else:
                    P, res = generate_from(body, name)
            out = fragments(P, res)
            LAST["P"], LAST["res"] = P, res
        except Exception as ex:                          # shown in the page
            import traceback
            traceback.print_exc()
            out = dict(ok=False, error=f"{type(ex).__name__}: {ex}")
        else:
            print(f"Done: {out['passed']} checks passed, {out['failed']} failed ({out['seconds']}s)",
flush=True)
        self._send(200, json.dumps(out), "application/json")


def serve():
    port = PORT
    for p in range(PORT, PORT + 20):
        try:
            srv = ThreadingHTTPServer(("127.0.0.1", p), Handler)
            port = p
```

```
            break
        except OSError:
            continue
    else:
        raise SystemExit("No free port found.")
    url = f"http://127.0.0.1:{port}/"
    print("Starting timetable generator...")
    print(f"Opening: {url}")
    webbrowser.open_new_tab(url)
    try:
        srv.serve_forever()
    except KeyboardInterrupt:
        print("\nStopped.")


def static_output(path):
    P, res = generate_from(Path(path))
    f = find_existing(HTML_NAMES)
    template = f.read_text(encoding="utf-8")
    section = ('<section id="timetable" class="view">' + timetable_inner(P, res) + "</section>")
    pat = re.compile(r'<section\s+id=["\']timetable["\'][^>]*>.*?</section>', re.I | re.S)
    if pat.search(template):
        page = pat.sub(lambda m: section, template, count=1)
    elif '<div id="out"></div>' in template:      # mLAC multi-page site: fill the Generate page
        page = template.replace('<div id="out"></div>', '<div id="out">' + timetable_inner(P, res) +
'</div>', 1)
        page = page.replace("</body>", "<script>location.hash='#generate';</script></body>", 1)
        page = page.replace("const out=$('out')", "const out=$('out')")
    else:
        raise ValueError('index.html has neither <section id="timetable"> nor the Generate page.')
    OUTPUT_FILE.write_text(page, encoding="utf-8")
    print(f"Timetable generated: {OUTPUT_FILE.name}")
    print(f"Checks: {res['passed']} passed, {res['failed']} failed")
```

```
    webbrowser.open_new_tab(OUTPUT_FILE.resolve().as_uri())

if __name__ == "__main__":
    ap = argparse.ArgumentParser(description="Smart timetable generator (Genetic Algorithm)")
    ap.add_argument("--dataset", help="generate Generated_Timetable.html directly from this workbook")
    a = ap.parse_args()
    if a.dataset:
        static_output(a.dataset)
    else:
        serve()
```

2. mLAC Smart Timetable.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Smart Timetable — mLAC · B.Sc Data Science</title>
  <link rel="stylesheet" href="styles.css">
  <style>
    /* Home page hero and call-to-action band (matches the approved home design) */
    .home-hero {
      text-align: center;
      padding: 64px 24px 24px;
    }
    .home-tag {
      display: inline-block;
      background: rgba(244,192,22,.22);
      color: #7A5C00;
      font-weight: 600;
      font-size: 12.5px;
```

```
  padding: 6px 14px;
  border-radius: 999px;
  margin-bottom: 20px;
}
.home-hero h1 {
  font-size: clamp(34px, 5.4vw, 60px);
  font-weight: 700;
  line-height: 1.12;
  max-width: 980px;
  margin: 0 auto 18px;
}
.home-hero h1 em { font-style: normal; color: #037BC0; }
.home-hero p {
  max-width: 620px;
  margin: 0 auto;
  color: #5A6275;
  font-size: 17.5px;
}
.home-band {
  margin-top: 40px;
  background: #1B2A4A;
  color: #fff;
  border-radius: 18px;
  padding: 40px 48px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 24px;
  flex-wrap: wrap;
}
.home-band h2 { font-size: 28px; font-weight: 600; margin: 0; }
.home-band p { margin: 6px 0 0; opacity: .92; }
```

```
.home-band .home-btn {
  background: #fff;
  color: #0A3F7D;
  font-weight: 600;
  padding: 12px 24px;
  border-radius: 999px;
  border: 0;
  cursor: pointer;
}
.home-band .home-btn:hover { filter: brightness(.95); }
#ttContent[hidden], #ttEmpty[hidden] { display: none; }
.sidebar-logo {
  width: 52px;
  height: auto;
  border-radius: 0;
  flex: none;
  background: none;
  border: 0;
}
body.home-active {
  background:
    linear-gradient(rgba(16,23,40,.80), rgba(16,23,40,.80)),
    url(college.jpg) center 35% / cover no-repeat fixed;
  min-height: 100vh;
}
body.home-active .app,
body.home-active .main {
  background: transparent !important;
}
#overview .overview-header {
  display: flex;
  flex-direction: column;
```

```
  align-items: center;
  text-align: center;
  padding: 96px 24px 24px;
  color: #fff;
}
#overview .overview-header .view-head { text-align: center; }
#overview .overview-header h1,
#overview .overview-header .lede { color: #fff; }
#overview .overview-header .eyebrow { color: #F4C016; }
#overview .overview-header .lede { margin-left: auto; margin-right: auto; }
/* Professional sizing and spacing for the home page */
#overview .overview-header h1 {
  font-size: clamp(38px, 5.4vw, 64px);
  font-weight: 700;
  line-height: 1.12;
  margin: 0 0 20px;
}
#overview .overview-header .lede {
  font-size: 19px;
  line-height: 1.6;
  max-width: 720px;
}
#overview .home-band {
  margin: 120px auto 0;
  max-width: 1000px;
  padding: 48px 56px;
  border-radius: 20px;
  box-shadow: 0 20px 48px rgba(0,0,0,.25);
}
#overview .home-band h2 {
  font-size: 36px;
  font-weight: 700;
```

```
  line-height: 1.2;
  margin: 0 0 10px;
}
#overview .home-band p {
  font-size: 19px;
  line-height: 1.6;
  margin: 0;
}
#overview .home-band .home-btn {
  font-size: 18px;
  font-weight: 600;
  padding: 16px 36px;
  white-space: nowrap;
}
.tt-drop {
  border: 2px dashed var(--line, #C9CFDB);
  border-radius: 16px;
  padding: 48px 20px;
  text-align: center;
  margin: 16px 0;
  transition: .15s;
  cursor: default;
}
.tt-drop.is-dragging {
  border-color: #037BC0;
  background: rgba(3,123,192,.06);
}
.tt-drop-icon { color: #037BC0; margin-bottom: 8px; }
.tt-browse {
  color: #037BC0;
  font-weight: 600;
  text-decoration: underline;
```

```
    cursor: pointer;
   }
  </style>
</head>
<body class="home-active">
<div class="app">

  <aside class="sidebar">
   <div class="brand">
    <img src="logo.png" alt="mLAC logo" class="sidebar-logo">
    <div class="brand-text">
     <strong>Smart Timetable</strong>
     <span>mLAC · Computer Science Dept.</span>
    </div>
   </div>

   <nav class="nav">
    <button class="nav-link is-active" data-target="overview">
     <span class="nav-dot"></span> Home
    </button>
    <button class="nav-link" data-target="about-project">
     <span class="nav-dot"></span> About the project
    </button>
    <button class="nav-link" data-target="timetable">
     <span class="nav-dot"></span> Timetable
    </button>
   </nav>

   <div class="sidebar-foot">
    <p><em>Developed by Harshitha N, Harshitha, Gagana S</em></p>
   </div>
  </aside>
```

```
<main class="main">
  <!-- HOME -->
  <section id="overview" class="view is-active">
    <div class="overview-header">
      <header class="view-head">
        <p class="eyebrow">B.Sc Data Science · Computer Science Department</p>
        <h1>Empowering faculty. Organising every week.</h1>
        <p class="lede">The mLAC Smart Timetable turns one spreadsheet into a clash-free week for every semester, with labs, classrooms and faculty all kept apart.</p>
      </header>
    </div>

    <div class="home-band">
      <div>
        <h2>Ready to build this semester's timetable?</h2>
        <p>Upload your dataset and see every semester laid out in seconds.</p>
      </div>
      <button class="home-btn" data-target="timetable">Generate Timetable</button>
    </div>
  </section>

  <!-- ABOUT THE PROJECT -->
  <section id="about-project" class="view">
    <header class="view-head">
      <p class="eyebrow">About the project</p>
      <h1>Smart AI-Based Automatic Timetable Generator</h1>
      <p class="lede">Built for the Department of Computer Science.</p>
    </header>

    <div class="panel">
      <div class="panel-head"><h2>What the project does</h2></div>
      <p>
```

```
        Preparing a college timetable by hand means juggling faculty, classrooms, laboratories
        and batches across several semesters. This project does it automatically from a single
        dataset, and checks that nothing clashes.
      </p>
    </div>
    <div class="panel">
      <div class="panel-head"><h2>How it works</h2></div>
      <table class="data-table">
        <thead>
          <tr><th>Step</th><th>What happens</th></tr>
        </thead>
        <tbody>
          <tr><td>1. Read the workbook</td><td>Academic, Resource, Batches, Faculty and Slot
sheets are read and cleaned, and problem rows are flagged.</td></tr>
          <tr><td>2. Plan labs, then theory</td><td>A Genetic Algorithm places lab blocks first,
pairs faculty, then places every theory period.</td></tr>
          <tr><td>3. Validate &amp;amp; export</td><td>Every rule is re-checked from scratch, then the
master and semester timetables are shown and exported to Excel.</td></tr>
        </tbody>
      </table>
    </div>

    <div class="panel">
      <div class="panel-head"><h2>Technologies used</h2></div>
      <p>Python · Genetic Algorithm · pandas · openpyxl · HTML · CSS · JavaScript</p>
    </div>
  </section>
  <!-- TIMETABLE -->
  <section id="timetable" class="view">
    <header class="view-head">
      <p class="eyebrow">Timetable</p>
      <h1>Master timetable</h1>
```

```
    <p class="lede">Every semester in one place.</p>
  </header>

  <div id="ttEmpty" class="panel">
    <p>Your timetable will appear here once you upload a dataset.</p>
    <div class="tt-drop" id="ttDrop">
      <div class="tt-drop-icon" aria-hidden="true">
        <svg viewBox="0 0 48 48" width="34" height="34">
          <path d="M24 6v24M24 6l-9 9M24 6l9 9" fill="none" stroke="currentColor" stroke-width="3" stroke-linecap="round" stroke-linejoin="round"/>
          <path d="M8 32v6a4 4 0 0 0 4 4h24a4 4 0 0 0 4-4v-6" fill="none" stroke="currentColor" stroke-width="3" stroke-linecap="round" stroke-linejoin="round"/>
        </svg>
      </div>
      <p><strong>Drag your dataset here</strong> or
        <label for="ttFile" class="tt-browse">browse from your computer</label></p>
      <p class="dropzone-hint">.xlsx workbook with Academic, Resource, Batches, Faculty and Slot sheets</p>
      <input type="file" id="ttFile" accept=".xlsx,.xlsm,.xls" hidden>
    </div>
    <p class="fineprint" id="ttStatus"></p>
  </div>

  <div id="ttContent" hidden>
  <div class="tabs" role="tablist">
    <button class="tab is-active" data-sem="master">Master</button>
    <button class="tab" data-sem="g1">Semester 1</button>
    <button class="tab" data-sem="g3">Semester 3</button>
    <button class="tab" data-sem="g5">Semester 5</button>
  </div>

  <div id="ttTables"></div>
```

```
      </div>
    </section>
  </main>
</div>

<script>
  // Sidebar navigation and in-page buttons (data-target)
  const navLinks = document.querySelectorAll('.nav-link');
  const views = document.querySelectorAll('.view');

  function showView(id) {
    views.forEach(view => {
      view.classList.toggle('is-active', view.id === id);
    });
    navLinks.forEach(link => {
      link.classList.toggle('is-active', link.dataset.target === id);
    });
    document.body.classList.toggle('home-active', id === 'overview');
    window.scrollTo(0, 0);
  }
  document.querySelectorAll('[data-target]').forEach(el => {
    el.addEventListener('click', () => showView(el.dataset.target));
  });
  // Timetable tabs: Master shows all semesters, others show one
  const tabs = document.querySelectorAll('.tab');
  const tables = document.getElementById('ttTables');

  tabs.forEach(tab => {
    tab.addEventListener('click', () => {
      tabs.forEach(item => item.classList.remove('is-active'));
      tab.classList.add('is-active');
```

```
      const target = tab.dataset.sem;
      tables.querySelectorAll(':scope > .grid-wrap').forEach(grid => {
        grid.hidden = target !== 'master' && grid.id !== target;
      });
    });
  });
  </script>
</body>
</html>
```

3.styles.css

```
:root {
--ink: #14213D;
--ink-soft: #4B5468;
--paper: #FAF7F0;
--paper-line: #E4DFD0;
--card: #FFFFFF;
--amber: #C97D1F;
--amber-soft: #FBE8CC;
--teal: #2F6F62;
--teal-soft: #DDEDE8;
--warn: #B0442E;
--warn-soft: #F6E1DB;

--font-head: Calibri, "Segoe UI", Arial, sans-serif;
--font-body: Calibri, "Segoe UI", Arial, sans-serif;
--font-mono: Calibri, "Segoe UI", Arial, sans-serif;

--radius: 10px;
}
/* GLOBAL STYLES */
```

```
* {
  box-sizing: border-box;
  }

html,
body {
margin: 0;
padding: 0;
background: var(--paper);
color: var(--ink);
font-family: Calibri, "Segoe UI", Arial, sans-serif;
font-size: 16px;
line-height: 1.4;
}

button,
input,
select,
textarea {
font-family: Calibri, "Segoe UI", Arial, sans-serif;
cursor: pointer;
}

button {
font-size: inherit;
}

h1,
h2,
p,
span,
strong,
```

```
em,
th,
td,
label,
a {
font-family: Calibri, "Segoe UI", Arial, sans-serif;
}

h1,
h2 {
margin: 0;
}

/* APP LAYOUT */
.app {
display: grid;
grid-template-columns: 250px minmax(0, 1fr);
min-height: 100vh;
}

/* SIDEBAR */
.sidebar {
background: var(--ink);
color: #EAECF4;
padding: 28px 20px;
display: flex;
flex-direction: column;
gap: 36px;
position: sticky;
top: 0;
height: 100vh;
overflow-y: auto;
```

```
}

.brand {
display: flex;
align-items: center;
gap: 12px;
}

.brand-mark {
width: 38px;
height: 38px;
border-radius: 9px;
background: var(--amber);
color: var(--ink);
font-weight: 700;
font-size: 20px;
display: flex;
align-items: center;
justify-content: center;
flex-shrink: 0;
}

.brand-text {
display: flex;
flex-direction: column;
line-height: 1.25;
}

.brand-text strong {
font-size: 19px;
}
```

```
.brand-text span {
font-size: 14px;
color: #9AA2C0;
}

.nav {
display: flex;
flex-direction: column;
gap: 4px;
}

.nav-link {
display: flex;
align-items: center;
gap: 10px;
background: none;
border: none;
color: #C4C9DE;
text-align: left;
padding: 10px 12px;
border-radius: 8px;
font-size: 16px;
transition: background 0.15s ease, color 0.15s ease;
}

.nav-link:hover {
background: rgba(255, 255, 255, 0.06);
color: #FFFFFF;
}

.nav-link.is-active {
background: rgba(201, 125, 31, 0.18);
```

```
color: #FFFFFF;
}

.nav-dot {
width: 6px;
height: 6px;
border-radius: 50%;
background: #5A6285;
flex-shrink: 0;
}

.nav-link.is-active .nav-dot {
background: var(--amber);
}

.sidebar-foot {
margin-top: auto;
border-top: 1px solid rgba(255, 255, 255, 0.1);
padding-top: 16px;
}

.sidebar-foot p {
font-size: 14px;
color: #C4C9DE;
margin: 0;
line-height: 1.5;
}

.sidebar-foot p:last-child em {
font-style: italic;
}
```

```
/* MAIN CONTENT */

.main {
padding: 40px 48px 70px;
width: 100%;
max-width: 1500px;
min-width: 0;
}

.view {
display: none;
}

.view.is-active {
display: block;
animation: fadeIn 0.25s ease;
}

@keyframes fadeIn {
from {
opacity: 0;
transform: translateY(4px);
}

to {
opacity: 1;
transform: translateY(0);
}
}

.view-head {
max-width: 640px;
```

```
margin-bottom: 28px;
}

.eyebrow {
font-size: 14px;
color: var(--amber);
margin: 0 0 10px;
letter-spacing: 0.02em;
}

.view-head h1 {
font-size: 34px;
font-weight: 600;
line-height: 1.2;
margin-bottom: 12px;
}

.lede {
color: var(--ink-soft);
font-size: 17px;
margin: 0;
}

/* OVERVIEW HEADER AND COLLEGE LOGO */

.overview-header {
display: flex;
justify-content: space-between;
align-items: flex-start;
gap: 24px;
margin-bottom: 32px;
}
```

```
.overview-header .view-head {
flex: 1;
min-width: 0;
margin-bottom: 0;
}

.college-logo {
display: block;
width: 115px;
height: 115px;
object-fit: contain;
flex-shrink: 0;
background: #FFFFFF;
border-radius: 50%;
}

/* STATISTICS */

.stat-strip {
display: grid;
grid-template-columns: repeat(3, minmax(0, 1fr));
gap: 16px;
margin-bottom: 32px;
}

.stat {
background: var(--card);
border: 1px solid var(--paper-line);
border-radius: var(--radius);
padding: 18px;
display: flex;
```

```
flex-direction: column;
min-width: 0;
}

.stat-value {
font-size: 32px;
font-weight: 600;
}

.stat-label {
font-size: 17px;
font-weight: 600;
margin-top: 2px;
}

.stat-sub {
font-size: 14px;
color: var(--ink-soft);
margin-top: 2px;
}

/* PANELS */

.split {
display: grid;
grid-template-columns: 1.1fr 0.9fr;
gap: 20px;
}

.panel {
background: var(--card);
border: 1px solid var(--paper-line);
```

```
border-radius: var(--radius);
padding: 20px;
margin-bottom: 20px;
min-width: 0;
}

.panel-head {
display: flex;
align-items: center;
justify-content: space-between;
gap: 12px;
margin-bottom: 14px;
}

.panel-head h2 {
font-size: 21px;
font-weight: 600;
}

.panel-tag {
font-size: 14px;
color: var(--ink-soft);
background: var(--paper);
border: 1px solid var(--paper-line);
padding: 3px 8px;
border-radius: 20px;
white-space: nowrap;
}

/* BUTTONS */

.btn {
```

```
border: none;
border-radius: 8px;
font-weight: 600;
}

.btn-quiet {
background: none;
color: var(--ink);
font-size: 15px;
padding: 0;
text-decoration: underline;
text-underline-offset: 3px;
text-decoration-color: var(--paper-line);
}

.btn-quiet:hover {
text-decoration-color: var(--amber);
}

.btn-primary {
background: var(--amber);
color: #FFFFFF;
font-size: 17px;
padding: 11px 20px;
}

.btn-primary:hover {
background: #B26D17;
}

.btn-primary:disabled {
opacity: 0.7;
```

```
cursor: progress;
}

/* TABLES */

.table-scroll {
overflow: auto;
border: 1px solid var(--paper-line);
border-radius: 8px;
max-height: 420px;
}

.data-table {
width: 100%;
border-collapse: collapse;
font-size: 14px;
}

.data-table th {
position: sticky;
top: 0;
background: var(--ink);
color: #EAECF4;
text-align: left;
padding: 10px 12px;
font-weight: 600;
font-size: 14px;
white-space: nowrap;
}

.data-table td {
padding: 9px 12px;
```

```
border-bottom: 1px solid var(--paper-line);
color: var(--ink);
}

.data-table tbody tr:nth-child(even) {
background: #FCFAF4;
}

.data-table tr.is-flagged td {
background: var(--warn-soft);
}

.cell-empty {
font-size: 13px;
color: var(--warn);
}

/* FILE UPLOAD AREA */

.dropzone {
border: 2px dashed var(--paper-line);
border-radius: var(--radius);
background: var(--card);
padding: 36px 20px;
text-align: center;
margin-bottom: 20px;
transition: border-color 0.15s ease, background 0.15s ease;
}

.dropzone.is-dragging {
border-color: var(--amber);
background: var(--amber-soft);
```

```
}

.dropzone-inner {
display: flex;
flex-direction: column;
align-items: center;
gap: 6px;
}

.dropzone-icon {
color: var(--amber);
margin-bottom: 6px;
}

.link-btn {
background: none;
border: none;
color: var(--amber);
font-weight: 600;
padding: 0;
text-decoration: underline;
text-underline-offset: 2px;
font-size: inherit;
}

.dropzone-hint {
font-size: 14px;
color: var(--ink-soft);
margin: 0;
}

.file-status {
```

```
font-size: 14px;
color: var(--teal);
min-height: 16px;
margin: 6px 0 0;
overflow-wrap: anywhere;
}

.upload-actions {
display: flex;
align-items: center;
flex-wrap: wrap;
gap: 14px;
margin-bottom: 32px;
}

/* SEMESTER TABS */

.tabs {
display: flex;
flex-wrap: wrap;
gap: 6px;
margin-bottom: 18px;
border-bottom: 1px solid var(--paper-line);
}

.tab {
background: none;
border: none;
padding: 10px 4px;
margin-right: 18px;
font-size: 16px;
font-weight: 600;
```

```
color: var(--ink-soft);
border-bottom: 2px solid transparent;
transform: translateY(1px);
}

.tab.is-active {
color: var(--ink);
border-bottom-color: var(--amber);
}

.sem-grid {
display: none;
}

.sem-grid.is-active {
display: block;
}

/* TIMETABLE GRID */
.grid-wrap {
border: 1px solid var(--paper-line);
border-radius: var(--radius);
overflow: auto;
background: var(--card);
margin-bottom: 12px;
}

.tt-grid {
width: 100%;
border-collapse: collapse;
table-layout: fixed;
min-width: 880px;
```

```
}

.tt-grid th {
background: var(--ink);
color: #EAECF4;
font-size: 13px;
font-weight: 600;
text-align: left;
padding: 8px 6px;
white-space: normal;
overflow-wrap: anywhere;
}

.tt-grid th:first-child,
.tt-grid td:first-child {
width: 78px;
}

.tt-grid td {
border: 1px solid var(--paper-line);
padding: 5px;
vertical-align: top;
overflow-wrap: anywhere;
}

.tt-grid td.tt-time {
font-size: 13px;
color: var(--ink-soft);
background: #FCFAF4;
text-align: center;
vertical-align: middle;
}
```

```
/* TIMETABLE SUBJECT CHIPS */

.slot-chip {
display: inline-flex;
flex-direction: column;
gap: 2px;
width: 100%;
padding: 6px;
border-radius: 7px;
font-size: 13px;
font-weight: 600;
line-height: 1.3;
border-left: 3px solid transparent;
overflow-wrap: anywhere;
word-break: normal;
}

.slot-chip em {
font-style: normal;
font-weight: 600;
font-size: 12px;
line-height: 1.3;
color: #000080 !important;
overflow-wrap: anywhere;
}

.slot-chip small {
font-weight: 500;
font-size: 11px;
line-height: 1.3;
color: #000080 !important;
```

```
margin-top: 2px;
}

.slot-chip.theory {
background: var(--amber-soft);
border-left-color: var(--amber);
color: #5C3B0E;
}

.slot-chip.theory em {
color: #000080 !important;
}
.slot-chip.lab {
background: var(--teal-soft);
border-left-color: var(--teal);
color: #1D4A40;
}
.slot-chip.lab em {
color: #000080 !important;
}
.slot-chip.free {
background: transparent;
border: 1px dashed var(--paper-line);
color: #777364;
font-weight: 500;
text-align: center;
align-items: center;
}

/* RESPONSIVE DESIGN */
@media (max-width: 1100px) {
.main {
```

```
padding: 30px 26px 50px;
}

.stat-strip {
grid-template-columns: repeat(2, minmax(0, 1fr));
}
.split {
grid-template-columns: 1fr;
}
}
@media (max-width: 800px) {
.app {
grid-template-columns: 1fr;
}
.sidebar {
position: relative;
height: auto;
flex-direction: column;
align-items: stretch;
gap: 20px;
}

.nav {
flex-direction: row;
flex-wrap: wrap;
}
.sidebar-foot {
margin-top: 0;
}
.main {
padding: 26px 20px 45px;
}
```

```
}

@media (max-width: 600px) {
.overview-header {
gap: 12px;
}

.college-logo {
width: 75px;
height: 75px;
}

.view-head h1 {
font-size: 29px;
}

.overview-header .view-head {
min-width: 0;
}

.stat-strip {
grid-template-columns: 1fr 1fr;
gap: 10px;
}

.stat {
padding: 14px 12px;
}
.stat-value {
font-size: 28px;
}
```

```
.stat-label {
font-size: 15px;
}
.stat-sub {
font-size: 13px;
}

.panel {
padding: 16px 12px;
}

.nav-link {
padding: 9px 10px;
font-size: 14px;
}
.tabs {
gap: 4px;
}
.tab {
margin-right: 10px;
font-size: 15px;
}
.tt-grid {
min-width: 880px;
}
}
/* ACCESSIBILITY */
@media (prefers-reduced-motion: reduce) {
.view.is-active {
animation: none;
}
}
```

4.3 Testing

Testing Type: Manual Testing

Test Cases

Test Case	Description
Dataset Upload Test	Ensures that a valid Excel (.xlsx) file can be uploaded through the web interface.
Dataset Validation Test	Checks whether the dataset contains the required information.
=Theory Scheduling Test	Verifies theory scheduling without repeating the same subject on the same day.
Faculty Conflict Test	Ensures that a faculty member is not assigned to multiple classes simultaneously.
Classroom Conflict Test	Checks that a classroom is not allocated to multiple activities at the same time.
Timetable Generation Test	Verifies that the program generates the timetable successfully.
Web-Based Output Test	Ensures that the generated timetable is displayed in the browser.
Invalid Dataset Test	Checks the system's response to invalid, incomplete, or unsupported Excel files.
Timetable Display Test	Verifies the display of days, time slots, subjects, classrooms, semesters, and labs.

Table 4.1: Test Cases of the Smart AI-Based Automatic Timetable Generator

CHAPTER 6

RESULTS AND DISCUSSION

RESULTS AND DISCUSSION

The Smart AI-Based Automatic Timetable Generator was successfully implemented using Python and a Genetic Algorithm-based scheduling approach. The system processes the manually prepared Excel dataset containing subjects, faculty, classrooms, batches, and time slots. Laboratory sessions are scheduled first, followed by theory classes. The generated timetable is checked using the implemented validation rules to identify faculty, classroom, and scheduling conflicts. The Python program generates the final timetable in HTML format, which is displayed through the web interface.

The website provides a home page, an About the Project page, and a Timetable section for uploading the Excel dataset. After successful generation, the **Master Timetable** displays the combined schedule of all configured semesters, while the **Semester-Wise View** allows users to view Semester 1, Semester 3, and Semester 5 separately. The results demonstrate that the system automates timetable generation, reduces manual effort, and provides an organized way to view academic schedules through a web browser.

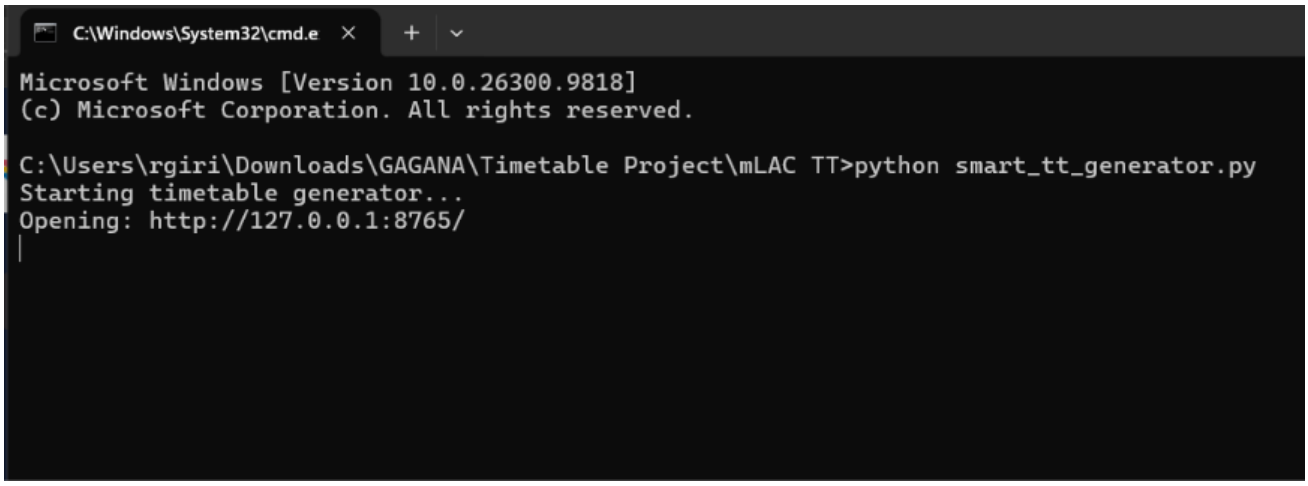


Figure 6.1: Execution of the Python Timetable Generator

The command prompt shows the execution of the Python program. It starts the timetable generator and launches the local web server.

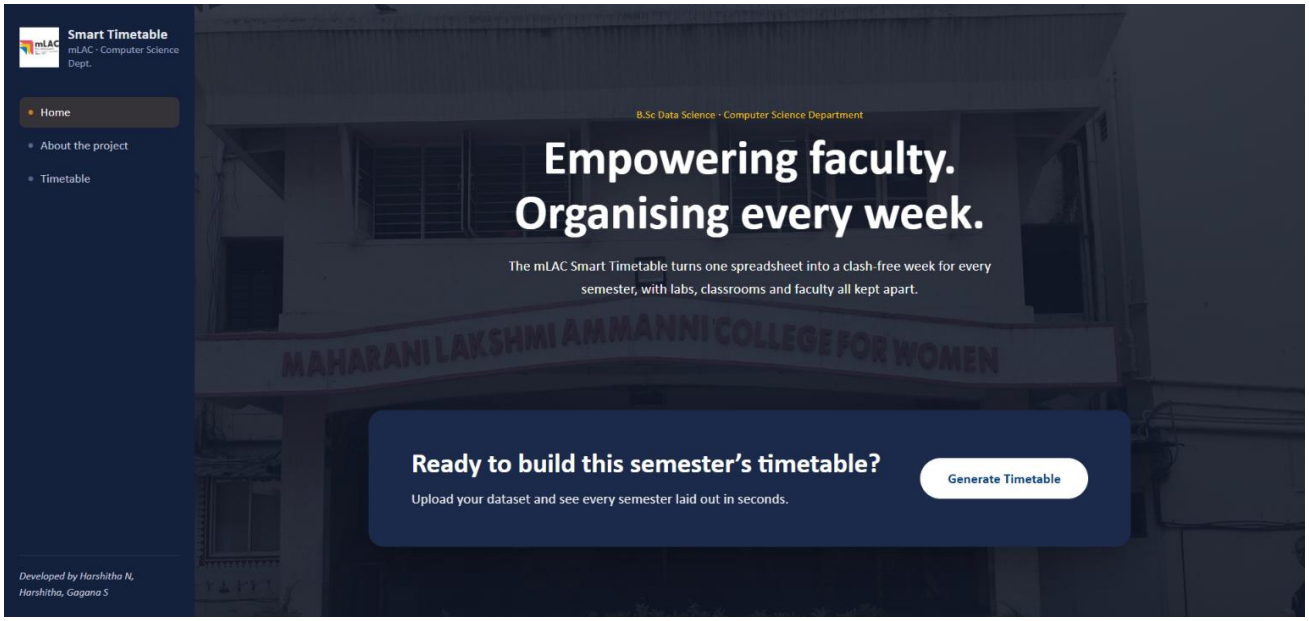


Figure 6.2: Home Page

The home page introduces the timetable generation system. It provides navigation to the project details and timetable generation section.

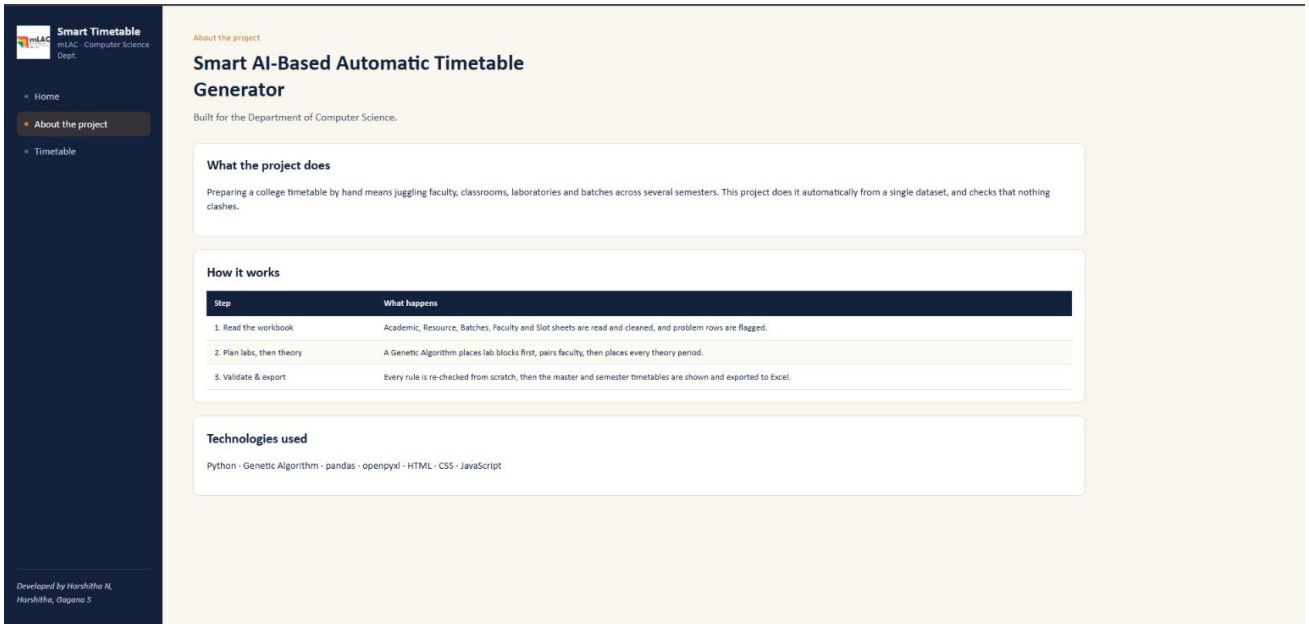


Figure 6.3: About the Project Page

This page explains the purpose of the project, its working process, and the technologies used for timetable generation.

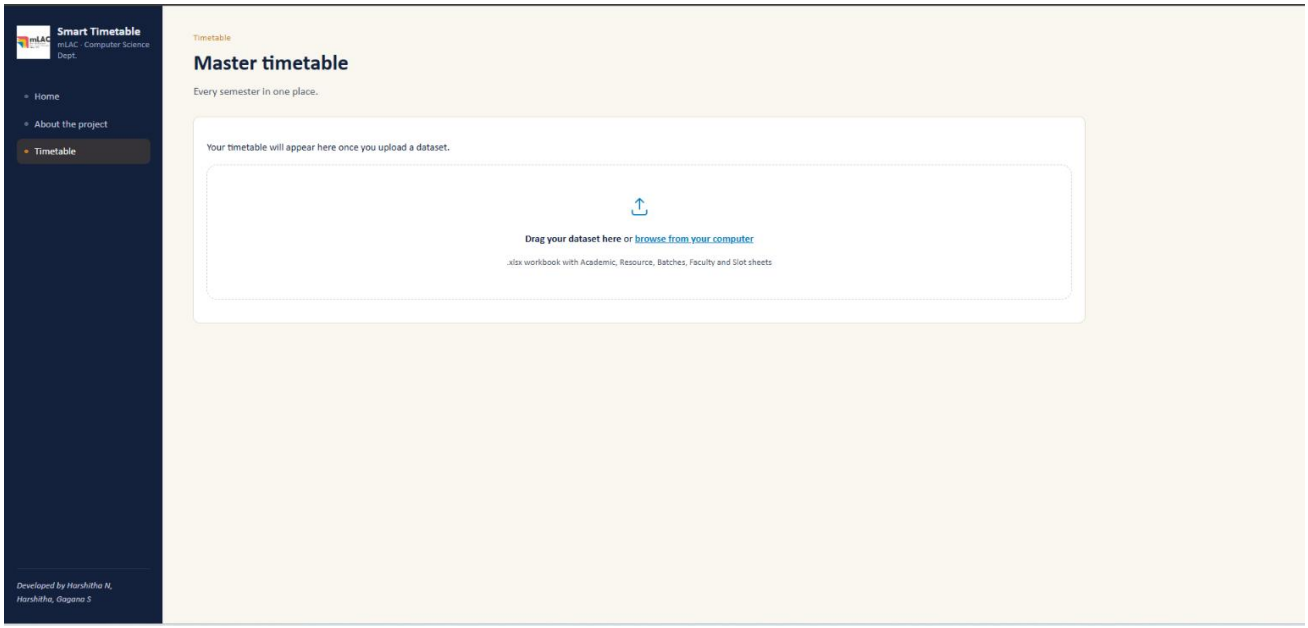


Figure 6.4: Timetable Dataset Upload Page

This page allows users to upload the required Excel dataset. The uploaded workbook is used as input for generating the timetable.

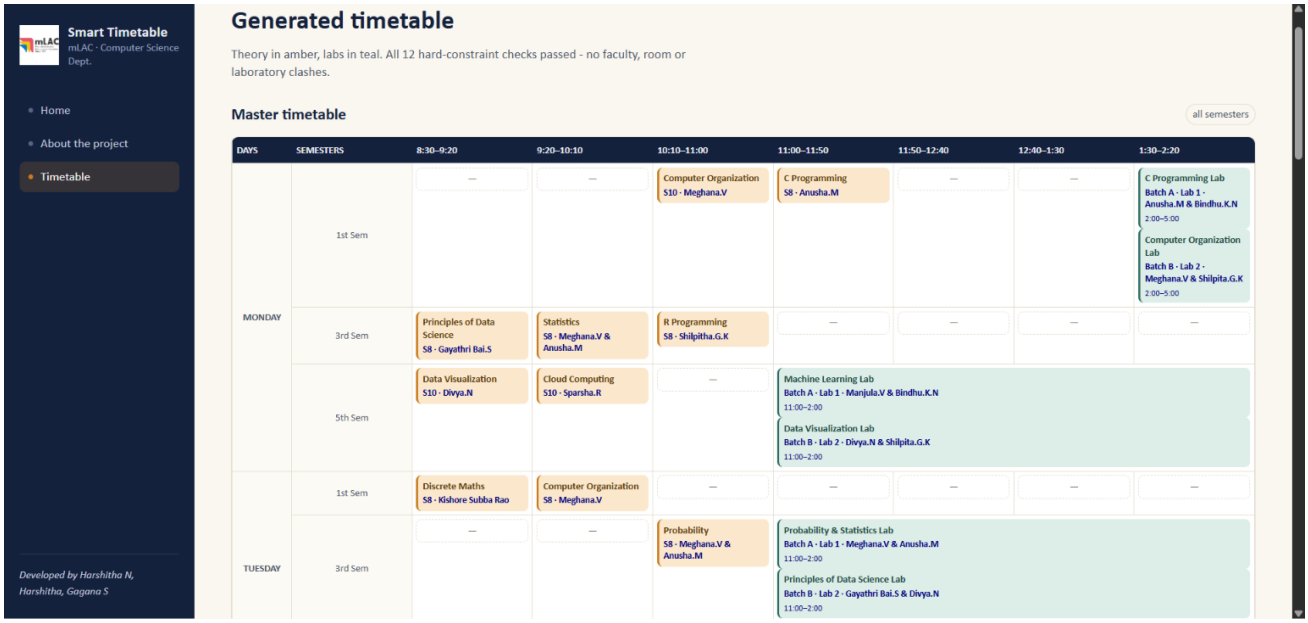


Figure 6.5: Generated Master Timetable Display

This page displays the generated master timetable containing all configured semesters. It presents theory classes and laboratory sessions according to days and time slots.

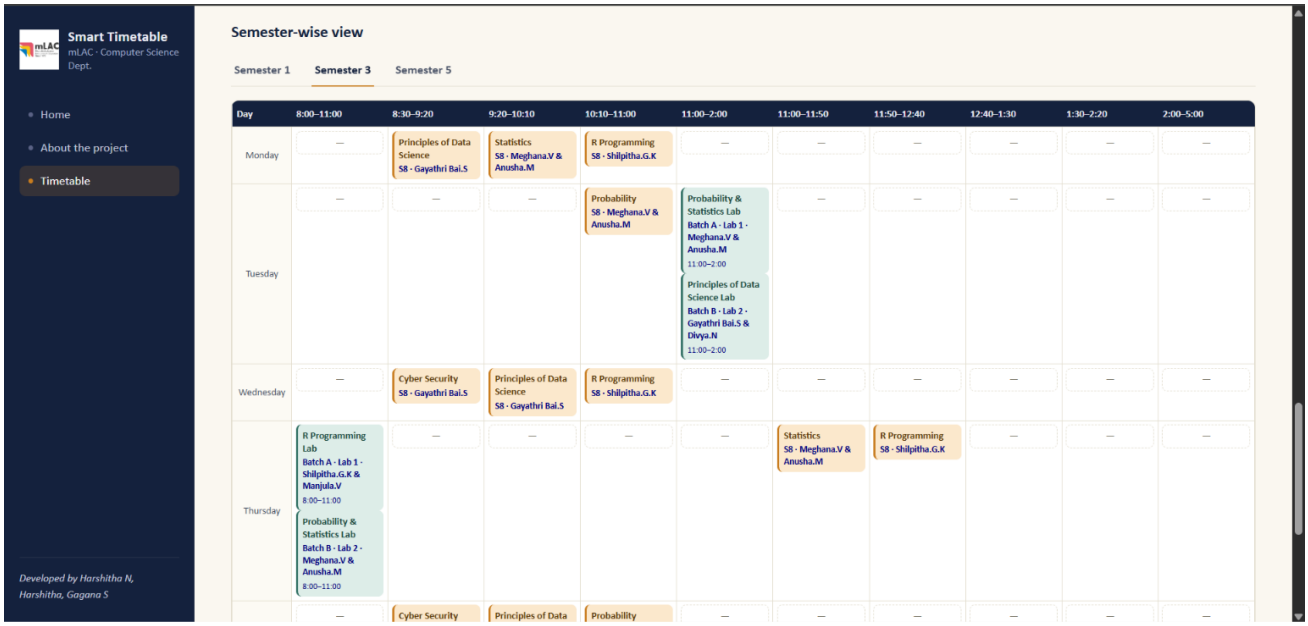


Figure 6.6: Semester-Wise Timetable Display

This page displays the timetable separately for each semester. Users can switch between semester tabs to view their individual schedules.

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENT

CONCLUSION

The Smart AI-Based Automatic Timetable Generator is developed to automate academic timetable preparation using Python and a Genetic Algorithm-based scheduling approach. It uses a manually prepared Excel dataset containing subject details, faculty information, classrooms, student batches, working days, and available time slots. The system processes this information using Python and Pandas, schedules laboratory sessions first, followed by theory classes, and applies the defined scheduling constraints to reduce faculty clashes, classroom conflicts, and overlapping sessions.

After scheduling, the system validates the generated timetable and converts it into HTML format for display through the web interface. It provides a home page, project information, dataset upload facility, combined Master Timetable, and Semester-Wise View. The project reduces manual effort, saves time, and improves the organization of academic resources. It also provides a foundation for future enhancements such as automatic rescheduling, real-time updates, user authentication, and multi-department support.

FUTURE ENHANCEMENTS

1. **Real-Time Faculty Availability:** Incorporate faculty availability, leave details, and schedule changes to improve timetable allocation.
2. **Automatic Timetable Rescheduling:** Introduce automatic timetable modification when faculty members are absent, classrooms become unavailable, or classes need to be rescheduled.
3. **Student and Faculty Login:** Provide separate login facilities for students, faculty members, and administrators to access their respective timetable information.
4. **Advanced Genetic Algorithm:** Improve the optimization process to generate more balanced timetables with fewer scheduling conflicts and better resource utilization.
5. **Multi-Department Support:** Extend the system to generate timetables for multiple departments, semesters, faculty members, and student batches within a single application.

CHAPTER 8

REFERENCES

REFERENCES

1. Chen, M. C., Sze, S. N., Goh, S. L., Sabar, N. R., & Kendall, G. (2021). A survey of university course timetabling problem: Perspectives, trends and opportunities. *IEEE Access, 9*, 106515–106529.https://doi.org/10.1109/ACCESS.2021.3100613

2. Algethami, H., & Laesanklang, W. (2021). A mathematical model for course timetabling problem with faculty-course assignment constraints. *IEEE Access, 9*, 111666–111682. https://doi.org/10.1109/ACCESS.2021.3103495

3. Davison, M., Kheiri, A., & Zografos, K. G. (2025). Modelling and solving the university course timetabling problem with hybrid teaching considerations. *Journal of Scheduling, 28*(2), 195–215. https://doi.org/10.1007/s10951-024-00817-w

4. Steiner, E., Pferschy, U., & Schaerf, A. (2025). Curriculum-based university course timetabling considering individual course of studies.*Central European Journal of Operations Research, 33*(1), 277–314.https://doi.org/10.1007/s10100-024-00923-2

5. Müller, T., Rudová, H., & Müllerová, Z. (2025). Real-world university course timetabling at the International Timetabling Competition 2019. *Journal of Scheduling, 28*(2), 247–267. https://doi.org/10.1007/s10951-023-00801-w

6. Arias-Osorio, J., & Mora-Esquivel, A. (2020). A solution to the university course timetabling problem using a hybrid method based on genetic algorithms. *Dyna, 87*(215), 47–56. https://doi.org/10.15446/dyna.v87n215.85933

7. Thepphakorn, T., Sooncharoen, S., & Pongcharoen, P. (2021). Particle swarm optimisation variants and its hybridisation ratios for generating cost-effective educational course timetables. *SN Computer Science, 2*, Article 236. https://doi.org/10.1007/s42979-021-00652-2

8. Alnowaini, G., & Aljomai, A. A. (2021). Genetic algorithm for solving university course timetabling problem using dynamic chromosomes. In *2021 International Conference of

Technology, Science and Administration (ICTSA)* (pp. 1–6). IEEE. https://doi.org/10.1109/ICTSA52017.2021.9406539

9. Muklason, A., Nugroho, B. A., Riksakomara, E., Tyasnurita, R., Mahananto, F., Vinarti, R. A., & Nuriman, M. A. (2022). Flexible automated course timetabling system with lecturer preferences using hyper-heuristic algorithm. In *Proceedings of the International Conference on Advanced Information Science and Engineering* (pp. 1–8). ACM. https://doi.org/10.1145/3568231.3568273

10. Jing, Y. (2021). Design and implementation of intelligent course arrangement system in colleges based on JSP technology. In *2021 4th International Conference on Computer Science and Software Engineering (CSSE 2021)* (pp. 1–6). ACM. https://doi.org/10.1145/3494885.3494938

11. Rappos, E., Thiémard, E., Robert, S., & Hêche, J.-F. (2022). A mixed-integer programming approach for solving university course timetabling problems. *Journal of Scheduling, 25*(1), 59–75. https://doi.org/10.1007/s10951-021-00715-5

12. Wang, C., Zhang, Y., & Zhou, H. (2023). Improved genetic algorithm for educational timetabling using adaptive fitness functions. *Applied Intelligence, 53*(7), 9214–9233. https://doi.org/10.1007/s10489-022-03600-7

13. Gozali, A. A., & Fujimura, S. (2020). Solving university course timetabling problem using multi-depth genetic algorithm. *SHS Web of Conferences, 1*, Article 01004. https://doi.org/10.1051/shsconf/20207601004

14. Abayomi-Alli, A., Misra, S., Fernández-Sanz, L., Abayomi-Alli, O., & Edun, A. R. (2020). Genetic algorithm and Tabu search memory with course sandwiching (GATS_CS) for university examination timetabling. *Intelligent Automation & Soft Computing, 26*(3), 385–396. https://doi.org/10.32604/iasc.2020.013915

15. Abduljabbar, I. A., & Abdullah, S. M. (2022). An evolutionary algorithm for solving academic courses timetable scheduling problem. *Baghdad Science Journal, 19*(2), Article 11. https://doi.org/10.21123/bsj.2022.19.2.0399